

学习内容

CONTENTS



1

神经元与神经网络

2

BP神经网络

3

卷积神经网络

4

生成对抗网络

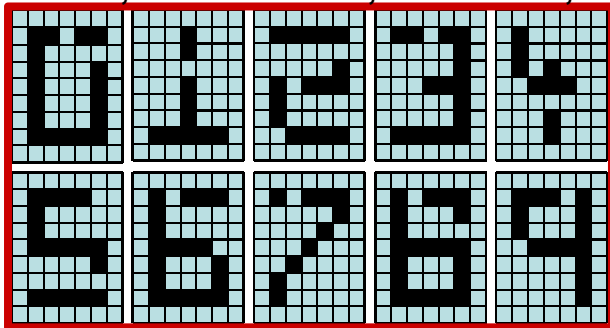
5

循环神经网络

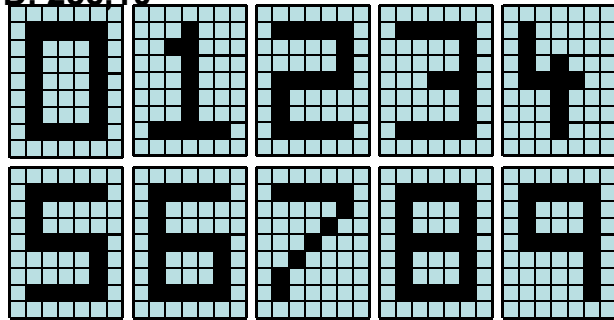
6

Transformer

✓当训练结束后，对如图所示测试数据进行测试。测试数据都有一个或者多个位丢失。识别手写体数字(16*16像素)，标签采用one-hot编码，那么输入层和输出层的神经元分别为多少？A.16,1 B. 256, 9 C. 16,10 D. 256,10



测试数据



训练数据

●测试结果表明：除了8以外，所有被测的数字都能够被正确地识别。

●对于数字8，神经网络的第6个结点的输出值为1，第8个结点的输出值为0.49，表明第8个测试样本是模糊的，可能是数字6，也可能是数字8，但也不完全确信是两者之一。

0	1	2	3	4	5	6	7	8	9
0	1	2	3	4	5	6	7	8	9
0	1	2	3	4	5	6	7	8	9
0	1	2	3	4	5	6	7	8	9
0	1	2	3	4	5	6	7	8	9
0	1	2	3	4	5	6	7	8	9
0	1	2	3	4	5	6	7	8	9
0	1	2	3	4	5	6	7	8	9
0	1	2	3	4	5	6	7	8	9
0	1	2	3	4	5	6	7	8	9



让计算机像人一样理解图像



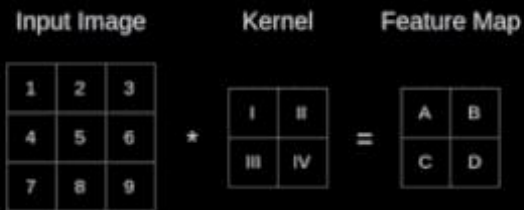
我们看到的

0	3	2	5	4	7	6	9	8
3	0	1	2	3	4	5	6	7
2	1	0	3	2	5	4	7	6
5	2	3	0	1	2	3	4	5
4	3	2	1	0	3	2	5	4
7	4	5	2	3	0	1	2	3
6	5	4	3	2	1	0	3	2
9	6	7	4	5	2	3	0	1
8	7	6	5	4	3	2	1	0

计算机看到的

局部感知（局部连接） + 参数(权重)共享 = 卷积

Connection Cutting Parameters Shering



假设无偏置
共4个参数

1

2

3

4

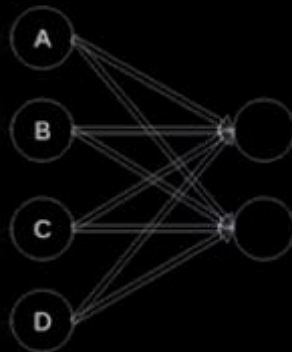
5

6

7

8

9



A: 1,2,4,5
B: 2,3,5,6
C: 4,5,7,8
D: 5,6,8,9



让计算机将语义概念相似的图像划分为同一类别

Manuel Lagunas & Elena Garces / Transfer Learning for Illustration Classification

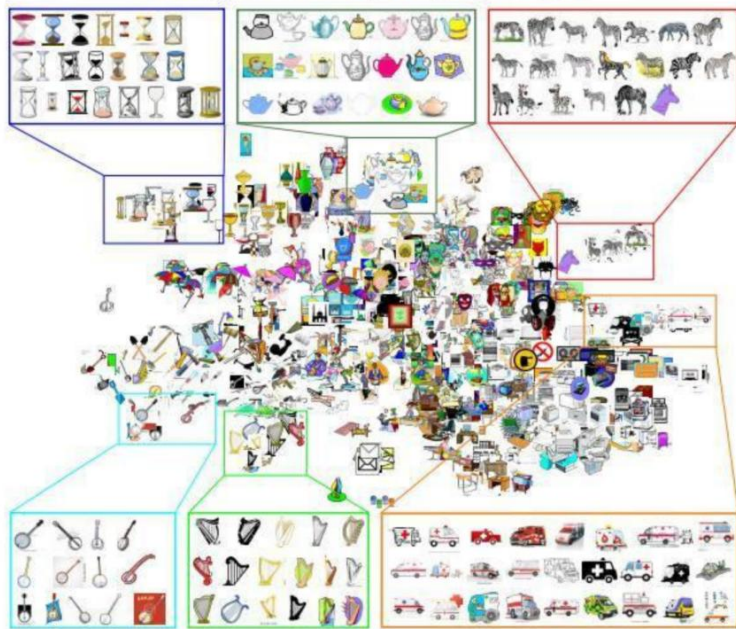


Figure 6: t-SNE algorithm on the image descriptors of the optimized network. The boxes show groups of images of the same class that have been grouped together after applying the algorithm. This shows us that the network is able to understand the low-level image characteristics and, consequently, that these image descriptors can be classified with a support vector machine obtaining great results.

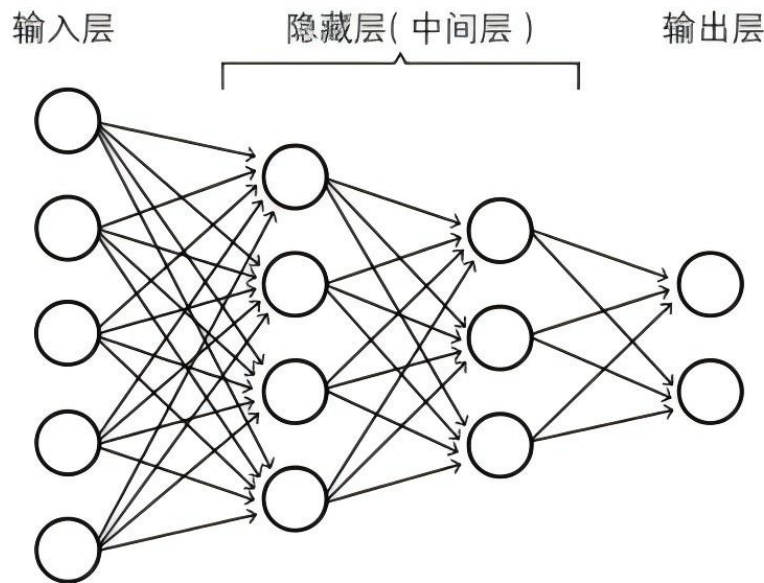
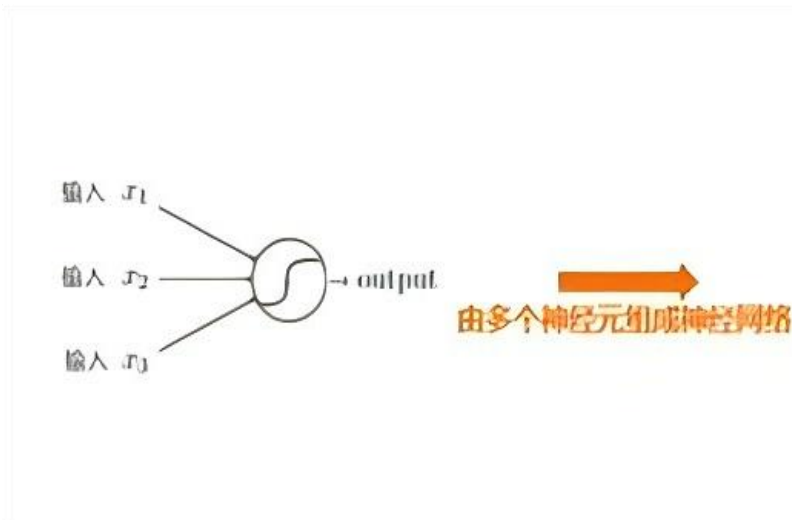
语义鸿沟(Semantic Gap)现象

- 又例如：不相似的视觉特性,相同的语义概念



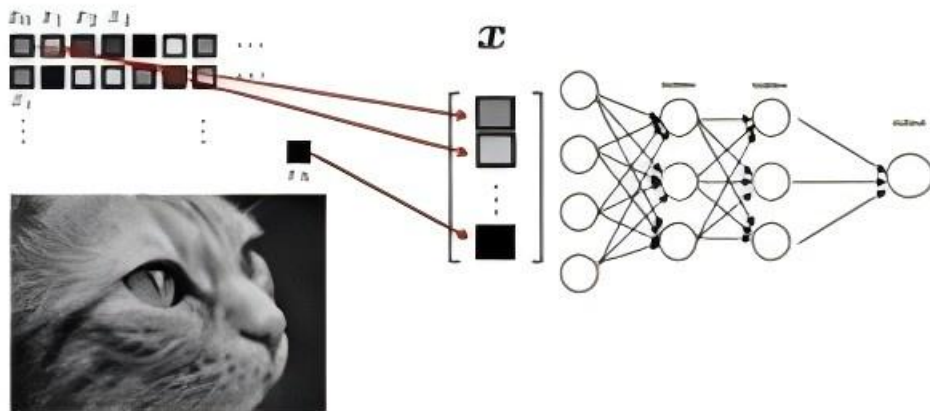
深层神经网络 (Deep Neural Networks, DNN) 是中间包含多个隐藏层的神经网络。

- 它由大量的、简单的处理单元 (称为神经元) 广泛地互相连接而形成的复杂网络系统,
- 它反映了人脑功能的许多基本特征, 是一个高度复杂的非线性动力学习系统。

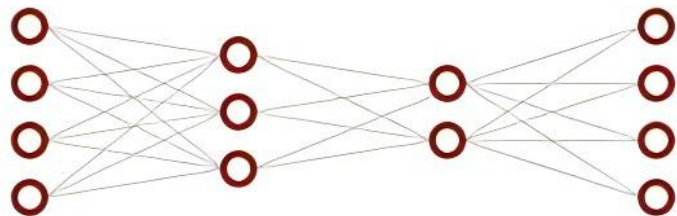


以图像分类为例,

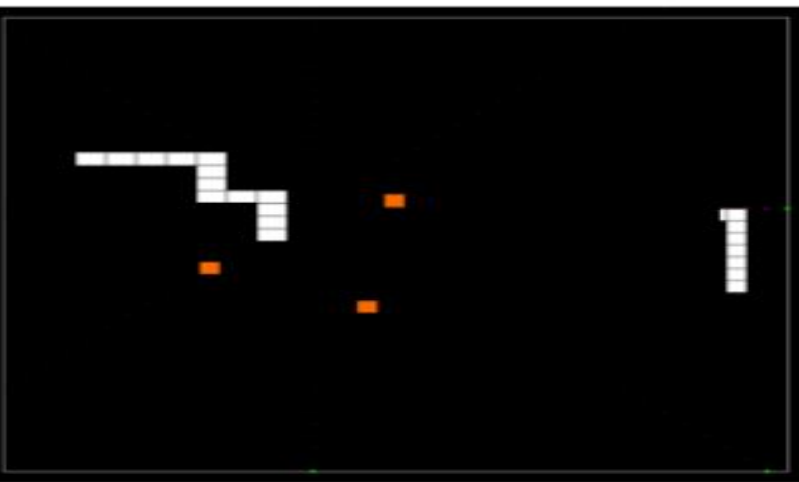
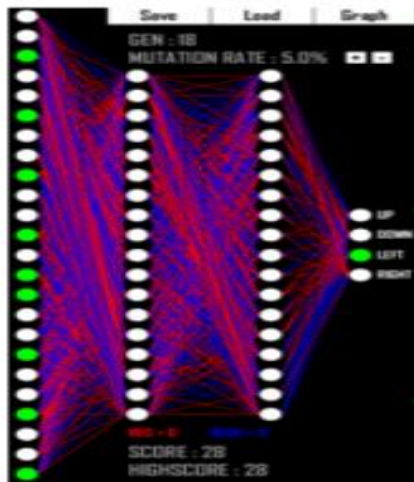
- 输入层是图像的像素数据。
- 隐藏层的层数以及每一层的神经元的个数就决定了这个神经网络的复杂程度。
- 一般来说层数越多, 每一层的神经元的个数越多, 这个网络越复杂, 它就能解决越复杂的问题。
- 输出层是分类的预测概率。



这是一只猫的概率是98%



控制飞机



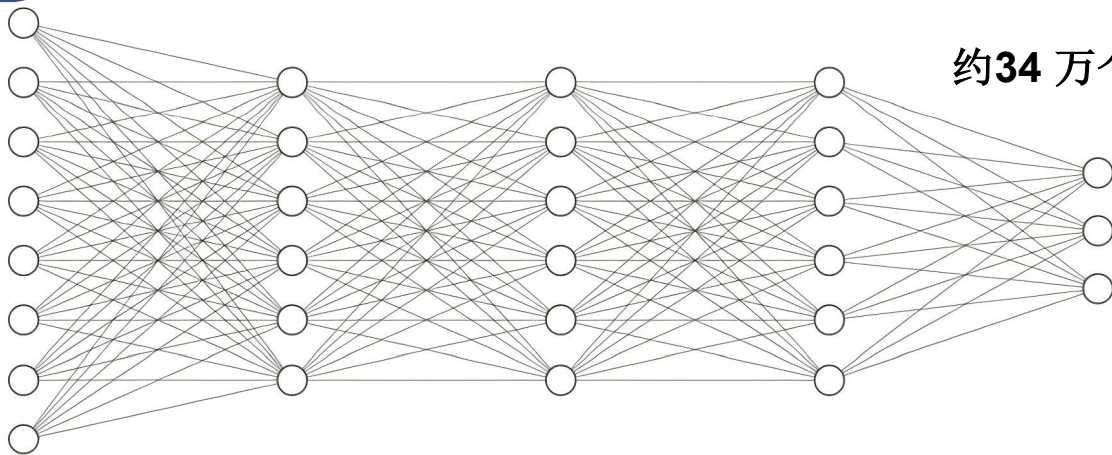
控制游戏中贪吃蛇的走向

8.3

深度神经网络: 全连接神经网络



约34 万个网络参数至少需要约 **1.34MB** 内存



输入: $[b, 784]$

隐藏层1: $[256]$

隐藏层2: $[256]$

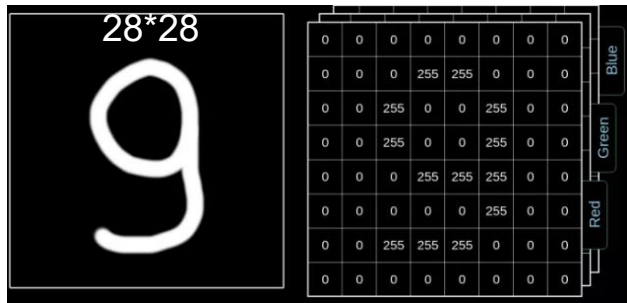
隐藏层3: $[256]$

输出: $[b, 10]$

层数	隐藏层 1	隐藏层 2	隐藏层 3	输出层
参数量	200960	65792	65792	2570
	$784 \times 256 + 256$			$256 \times 10 + 10$

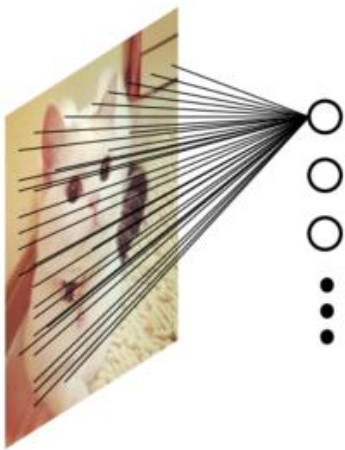
是否有必要全连接?

1989年, Yann LeCun 在手写邮政编码识别采用了一台 256KB 内存的计算机实现了算法 (LeNet系列卷积网络的雏形)。



似乎多层感知机+通用近似定理+BP学习算法可以解决所有问题？

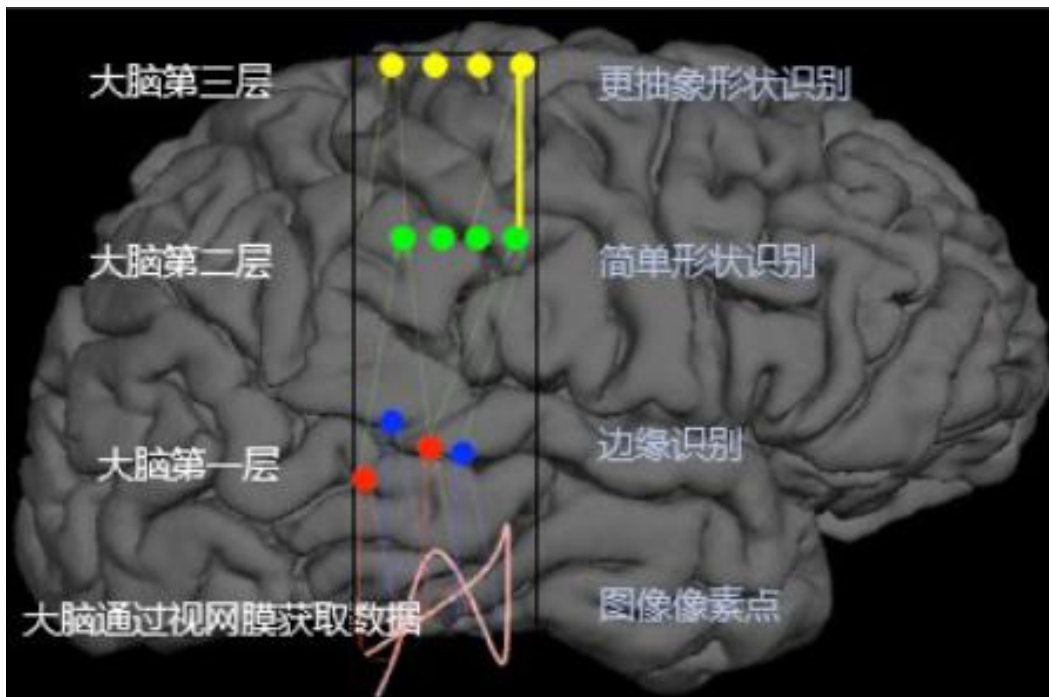
- 多层感知机在处理密集输入时**参数量爆炸**，利用当时的计算机难以模拟大规模神经网络
- 局部不变性特征
自然图像中的物体都具有**局部不变性特征**：尺度缩放、平移、旋转等操作不影响其语义信息。**全连接前馈网络很难提取这些局部不变特征**



处理一张 1000×1000 的图像，单个神经元的权重数量就达到100万！

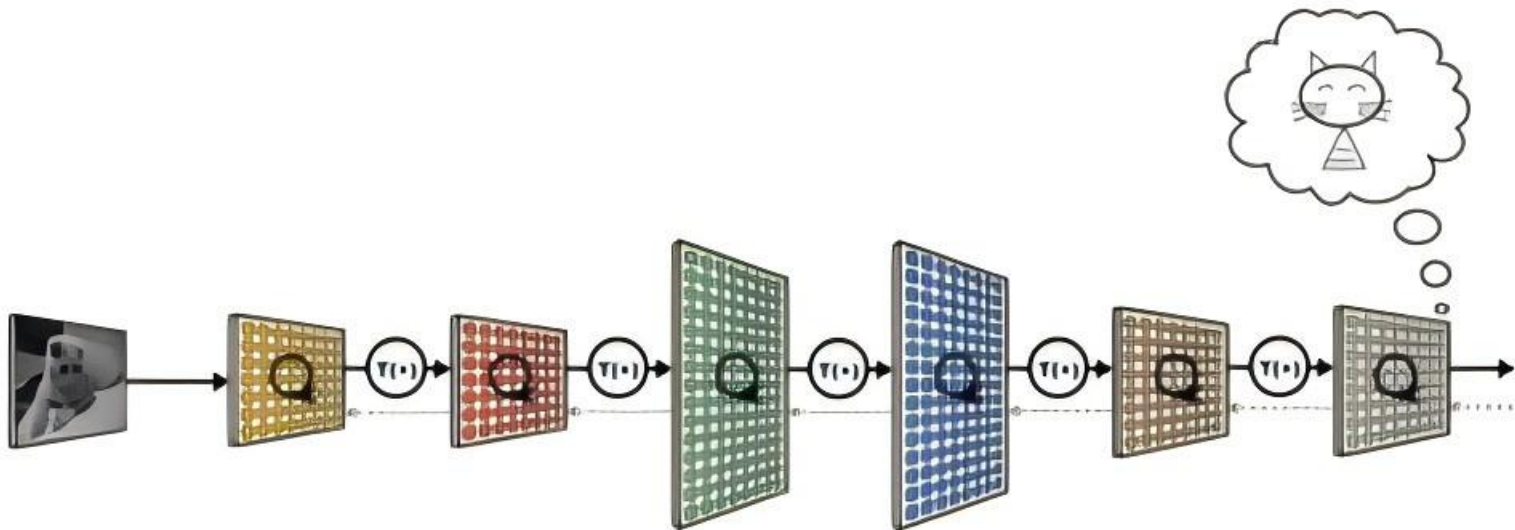


卷积神经网络(CNN)的诞生可追溯至日本学者福岛邦彦（Kunihiko Fukushima），他于1980年发明的Neocognitron（神经认知机）是CNN的雏形。

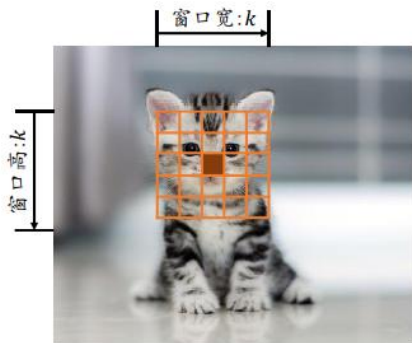


大脑在识别物体

神经网络有多层结构，也可以逐层提取特征，并归纳总结提取出物体的特征（比如认出一只猫）



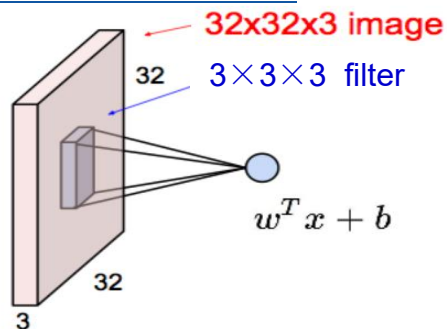
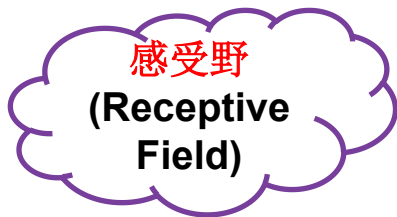
大脑结构与卷积神经网络识别物体的过程



特性1：图像中的像素是局部相关的



特性2：图像中的不同局部区域具有相同的统计性质



每个输出节点仅与感受野区域内
 $k \times k$ 个输入节点相连接(例如 $k=3$)

$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{bmatrix}$$

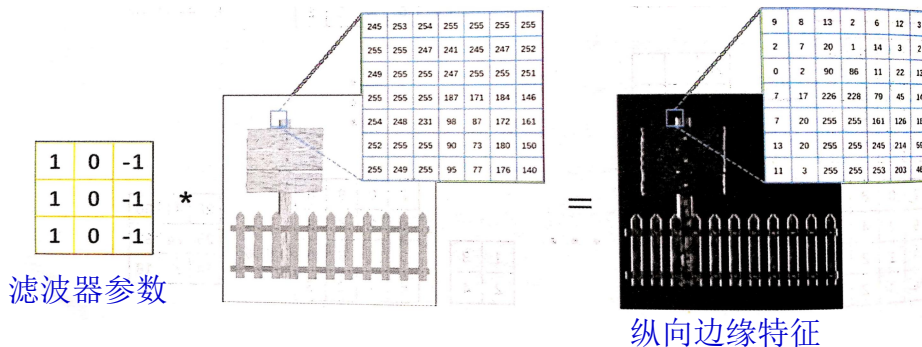
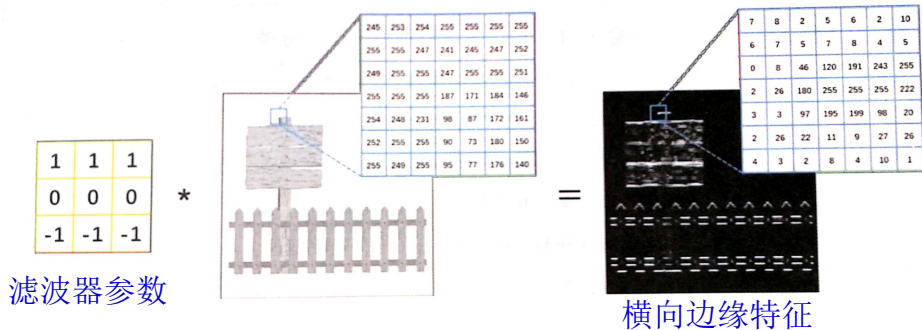
权值矩阵 $W_{3 \times 3}$ 示意图

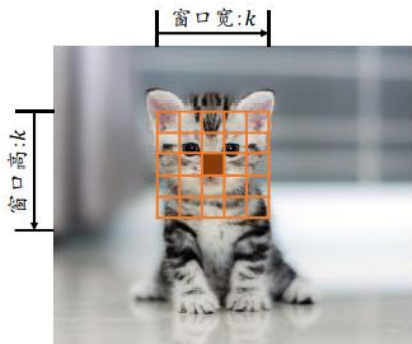
人眼感知从局部到全局
生物学依据：大脑中的视神经元仅受
环境中局部区域的刺激



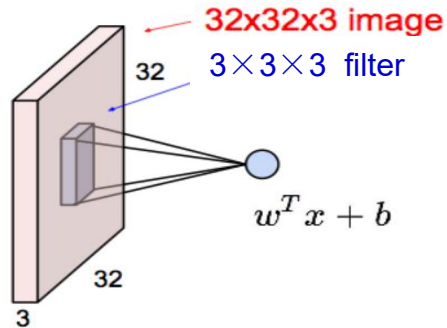
那就叫小苦吧 bilibili

不同的卷积核（也叫滤波器，filter）可以用来提取原始图像中的不同特征





感受野
(Receptive Field)



特性1：图像中的像素是局部相关的

每个输出节点仅与感受野区域内
 $k \times k$ 个输入节点相连接(例如 $k=3$)

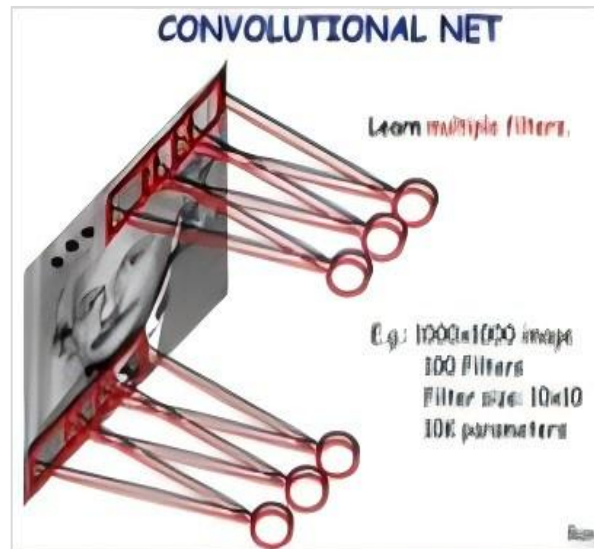
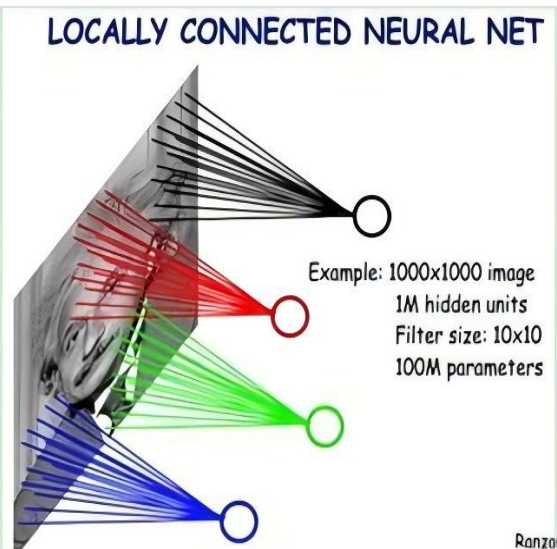
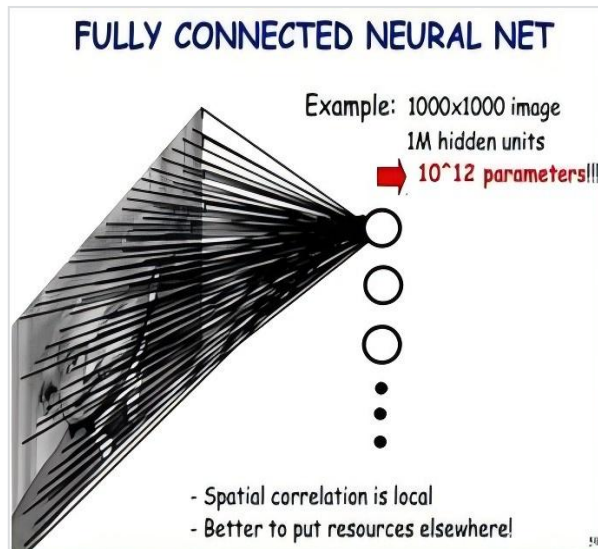


$$W = \begin{bmatrix} W_{11} & W_{12} & W_{13} \\ W_{21} & W_{22} & W_{23} \\ W_{31} & W_{32} & W_{33} \end{bmatrix}$$

权值矩阵 $W_{3 \times 3}$ 示意图

特性2：图像中的不同局部区域具有相同的统计性质

局部感知(局部连接)
+ 参数(权值)共享
= 卷积



全局感知:全
连接结构,
参数过多

人眼感知从局部到全
局生物学依据:大脑中
的视神经仅受环境
中局部区域的刺激

卷积神经网络结构特点?

- A、多层 B、前向
C、无反馈 D、全连接
E、局部连接

卷积神经网络中，感受野的含义是什么？

- ☐ A 卷积核的大小
- ☐ B 输入图像中单个像素对应输出特征图的区域大小
- ☐ C 特征图中的每个像素在原始输入图像上能够"看到"的区域大小
- ☐ D 网络最后一层对输入图像的整体覆盖范围

提交



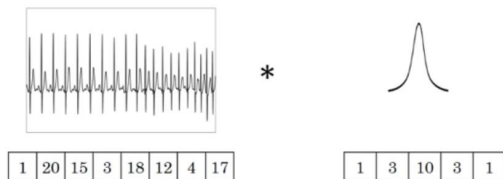
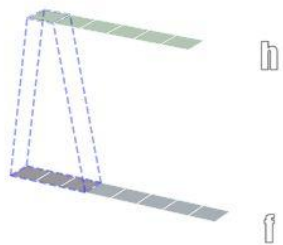
卷积神经网络中，感受野的含义是什么？

- ☐ A 卷积核的大小
- ☐ B 输入图像中单个像素对应输出特征图的区域大小
- ☒ C 特征图中的每个像素在原始输入图像上能够"看到"的区域大小
- ☐ D 网络最后一层对输入图像的整体覆盖范围

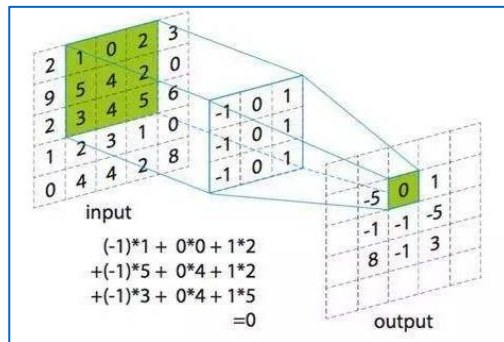
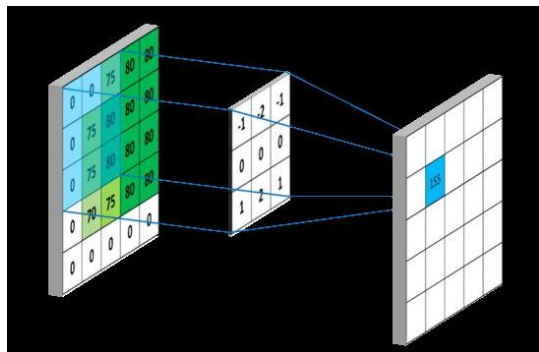
提交



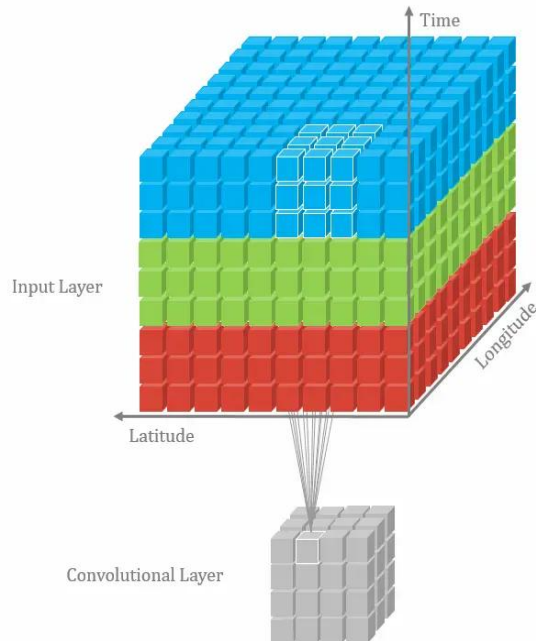
from torch.nn import Conv1d, Conv2d, Conv3d



一维卷积：自然语言处理，时间序列预测，音频信号处理



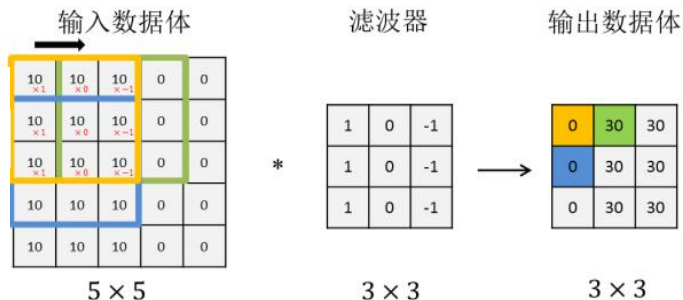
二维卷积：图像分类、目标检测、图像分割、特征提取（边缘检测、纹理识别等）



三维卷积：医学影像、视频处理，立体数据建模

from torch.nn import Conv1d, **Conv2d**, Conv3d

单通道卷积
(灰度图)

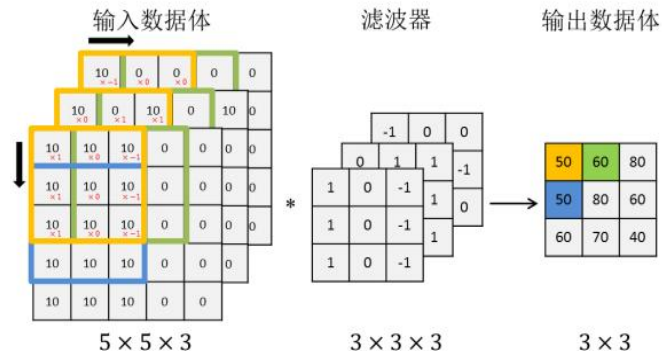


$$\begin{aligned}
 &10 \times 1 + 10 \times 0 + 10 \times (-1) + \\
 &10 \times 1 + 10 \times 0 + 10 \times (-1) + \\
 &10 \times 1 + 10 \times 0 + 10 \times (-1) = \\
 &0
 \end{aligned}$$

步长stride为1
偏置b为0

`torch.nn.Conv2d(in_channels=1,out_channels=1,kernel_size=(3,3),stride=1,bias = False)`

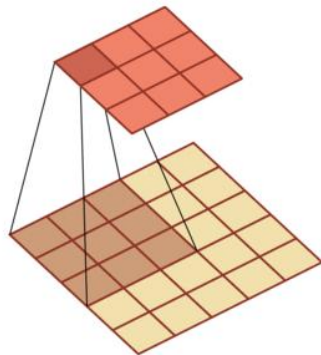
多通道卷积
(彩色图像)



`torch.nn.Conv2d(in_channels=3,out_channels=1,kernel_size=(3,3),stride=1,bias = False)`

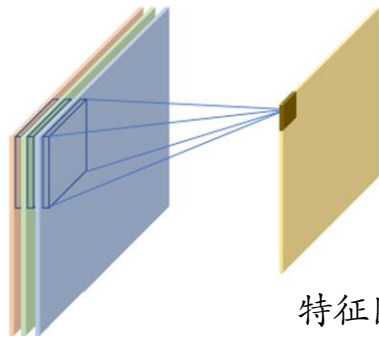
每个通道用一个滤波器卷积之后得到对应一个通道的输出，然后再进行信息的融合

特征图



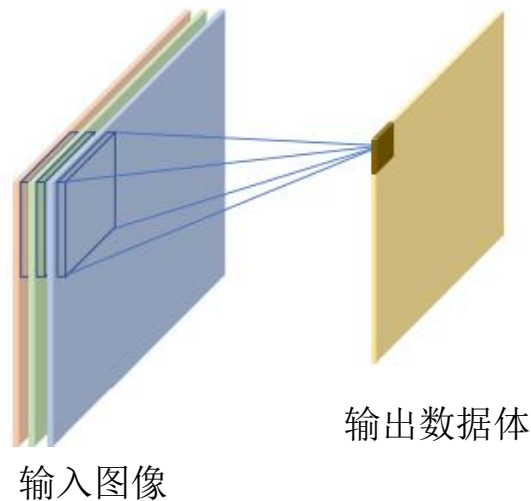
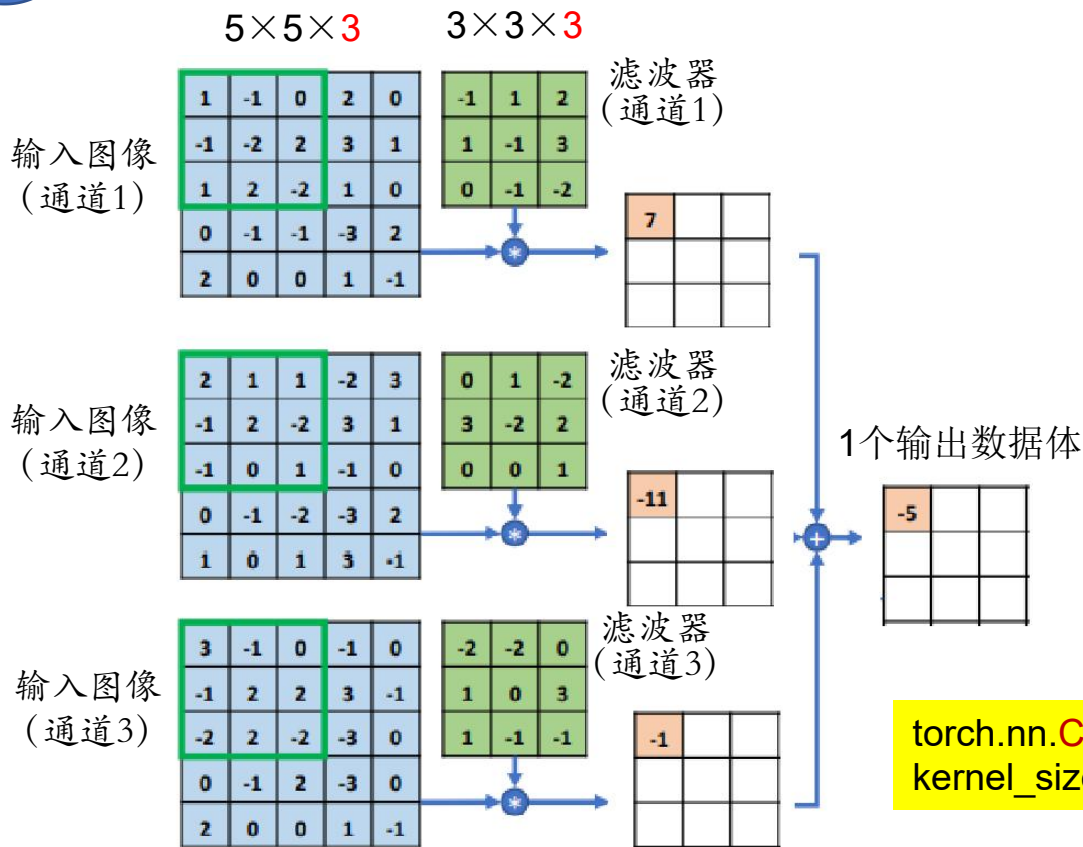
输入图像

卷积运算
p234-235



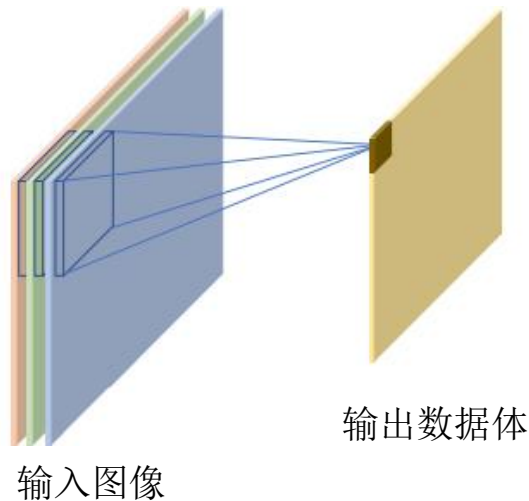
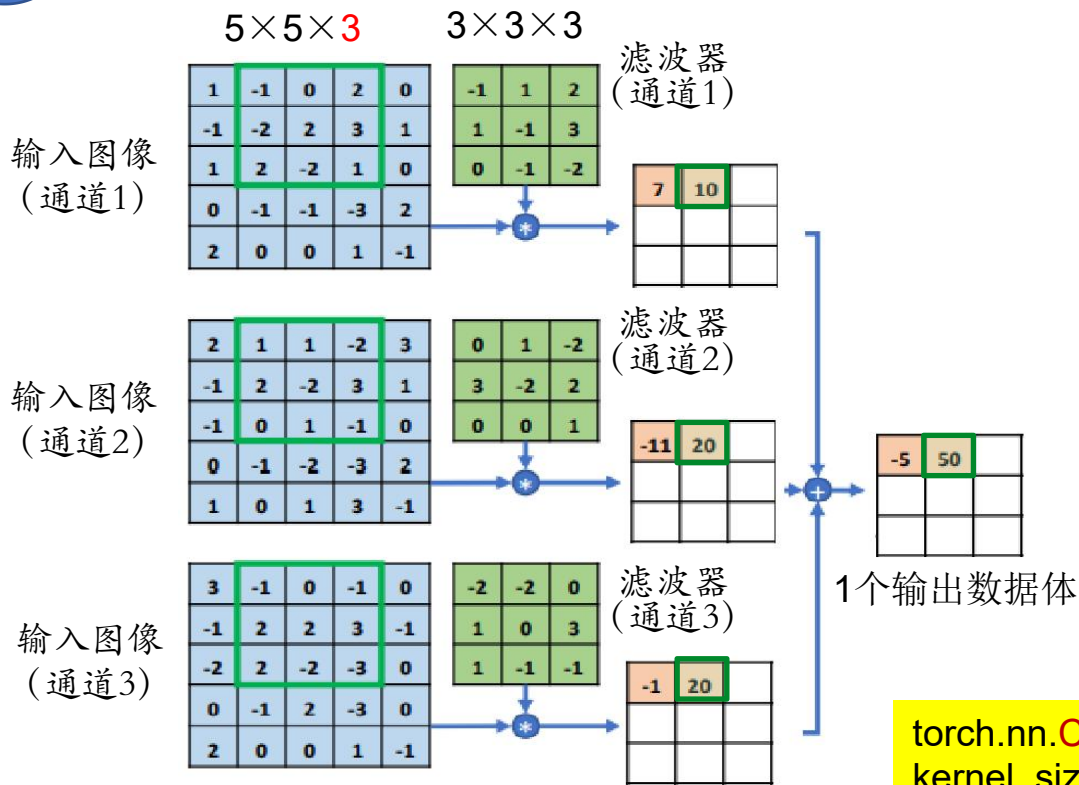
特征图
(Feature Map)

输入图像



```
torch.nn.Conv2d(in_channels=3,out_channels=1,
kernel_size=(3,3),stride=1, bias = False)
```

2维卷积（输入：3通道的5×5图像，采用3通道的3×3卷积核，stride=1），偏置**b**为0



```
torch.nn.Conv2d(in_channels=3,out_channels=1,
kernel_size=(3,3),stride=1, bias = False)
```

2维卷积（输入：3通道的5×5图像，采用3通道的3×3卷积核，stride=1），偏置**b**为0

Input Volume (+pad 1) (7x7x3)

x[:, :, 0]						
0	0	0	0	0	0	0
0	0	1	1	2	2	0
0	0	1	1	0	0	0
0	1	1	0	1	0	0
0	1	0	1	1	1	0
0	0	2	0	1	0	0
0	0	0	0	0	0	0
x[:, :, 1]						
0	0	0	0	0	0	0
0	1	1	1	2	0	0
0	0	2	1	1	2	0
0	1	2	0	0	2	0
0	0	2	1	2	1	0
0	2	0	1	2	0	0
0	0	0	0	0	0	0
x[:, :, 2]						
0	0	0	0	0	0	0
0	2	0	2	0	2	0
0	0	0	1	2	1	0
0	1	0	2	2	1	0
0	2	0	2	0	0	0
0	0	0	1	1	2	0
0	0	0	0	0	0	0

输入图像 5*5*3

Filter W0 (3x3x3)

w0[:, :, 0]		
1	1	-1
-1	0	1
-1	-1	0
w0[:, :, 1]		
-1	0	-1
0	0	-1
1	-1	0
w0[:, :, 2]		
0	1	0
1	0	1
0	-1	1
Bias b0 (1x1x1)		
b0[:, :, 0]		
1		

第1个卷积核

stride=2, padding=1

Filter W1 (3x3x3)

w1[:, :, 0]		
-1	-1	0
-1	1	0
-1	1	0
w1[:, :, 1]		
1	-1	0
-1	0	-1
-1	0	0
w1[:, :, 2]		
-1	0	1
1	0	1
0	-1	0
Bias b1 (1x1x1)		
b1[:, :, 0]		
0		

第2个卷积核

toggle movement

Output Volume (3x3x2)

o[:, :, 0]		
1	0	-3
-6	1	1
4	-3	1
o[:, :, 1]		
-1	-6	-4
-2	-3	-4
-1	-3	-3

第1个输出数据体

第2个输出数据体

输出

一个重要的原则：
滤波器（卷积核）和输入数据体必须具有相同通道数

输出数据体的
大小如何确定？

```
torch.nn.Conv2d(in_channels=3,out_channels=2,
kernel_size=(3,3),stride=2,padding=1,bias = True)
```

在卷积层中，最终输出特征图尺寸除了与输入图像尺寸($H \times W \times C$)相关以外，还与卷积核的尺寸($F \times F$)、填充量(p)和移动步长(s)相关。

输出特征图尺寸计算方法为：

- 输出高度 = $\lfloor (H - F + 2p) / s \rfloor + 1$
- 输出宽度 = $\lfloor (W - F + 2p) / s \rfloor + 1$
- 输出通道数 = K (卷积核个数)

$\lfloor \rfloor$ ：向下取整

$$(5 + 2 \times 0 - 3) / 1 + 1 = 3$$

卷积核 3×3 ，无填充，
步长为1

1x1	1x0	1x1	0	0
0x0	1x1	1x0	1	0
0x1	0x0	1x1	1	1
0	0	1	1	0
0	1	1	0	0

输入图像 5×5

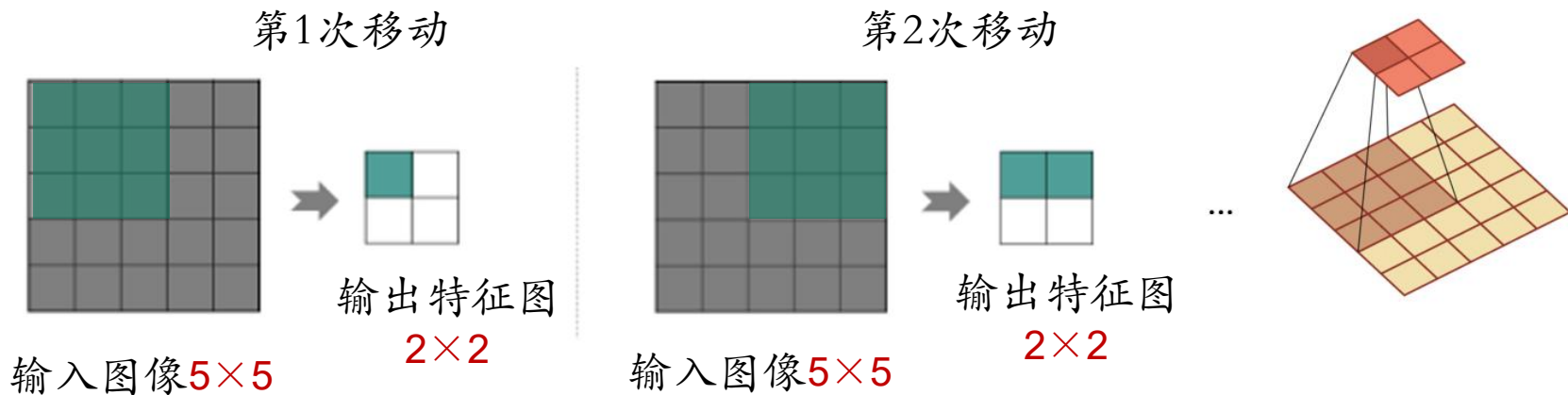
4		

输出特征图

3×3

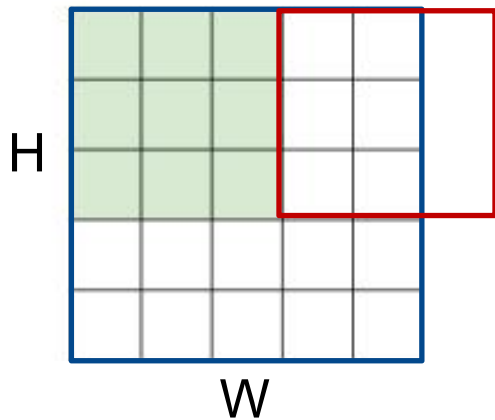
步长Stride表示卷积核每次的移动步长

在步长为2的情况下移动卷积核（大小： 3×3 ，无填充），移动过程如下图：



$$(5 + 2 \times 0 - 3) / 2 + 1 = 2$$

步长越大，输出数据体越小！

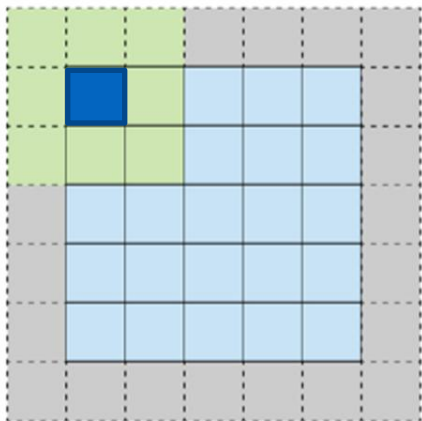


输入图像 5×5

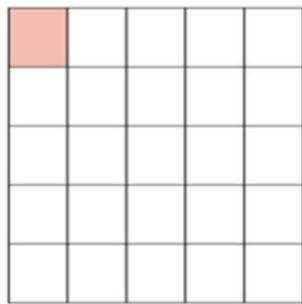
问题：假设输入数据体大小为 5×5 ，卷积核（滤波器）的大小为 3×3 ，步长stride为3，则输出数据体的大小为多少？

卷积计算后得到的特征图尺寸通常比原始图像的尺寸更小，可以在原始图像周围进行填充（padding）操作，可以增大卷积计算后得到的特征图尺寸。

卷积核 3×3 ，填充为1，步长为1



输入图像 5×5



特征图 5×5

Feature Map

• 填充 (Padding) :

- 在输入数据体边缘处填补特定元素
- 最常用的是使用0元素进行填充，即零填充
- 填充可以控制输出数据体的空间尺寸

Same 填充（步长 $s=1$ ，输入和输出的数据体大小相同）：

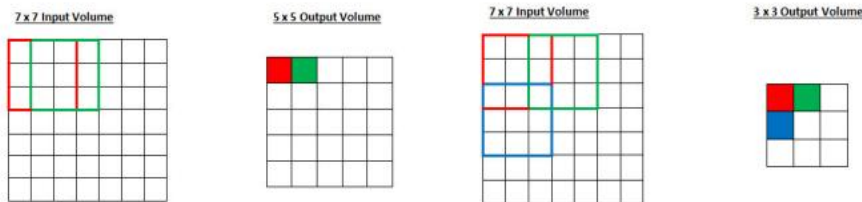
填充量 $p = (f - 1) / 2$

$$(5 + 2 \times 1 - 3) / 1 + 1 = 5$$

```
import torch.nn as nn
```

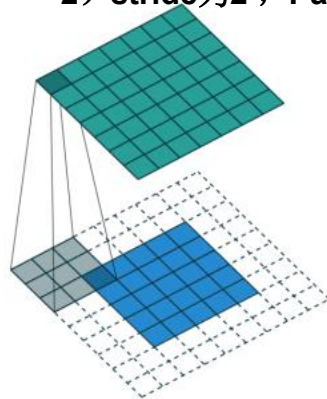
```
nn.Conv2d( in_channels=1, out_channels=1, kernel_size=3, stride=1, padding=1)
```

- **通道 (Channel) :**
 - 输出数据体的通道数量 (也称深度, Depth) , 即所使用的滤波器的数量
- **步长 (Stride) :**
 - 在滑动滤波器的时候, 平移的距离称为步长
 - 步长可以控制输出数据体在空间上的尺寸
- **填充 (Padding) :**
 - 在输入数据体边缘处填补特定元素
 - 最常用的是使用0元素进行填充, 即零填充
 - 填充可以控制输出数据体的空间尺寸
 - **保持边界信息**



1) stride为1, Padding为0

2) stride为2, Padding为0



3) stride为1, Padding为2

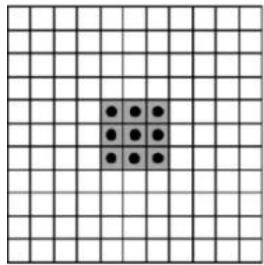
除了标准卷积, 还有空洞卷积、组卷积、深度可分离卷积、转置卷积等

$$H_{out} = \left\lfloor \frac{H_{in} + 2 \times padding[0] - dilation[0] \times (kernel_size[0] - 1) - 1}{stride[0]} + 1 \right\rfloor$$

⌊ ⌋: 若不能整除, 向下取整

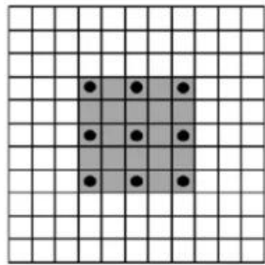
$$W_{out} = \left\lfloor \frac{W_{in} + 2 \times padding[1] - dilation[1] \times (kernel_size[1] - 1) - 1}{stride[1]} + 1 \right\rfloor$$

空洞卷积



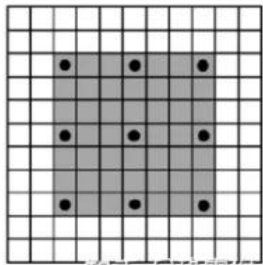
a)

Dilation=1
(卷积核 3×3)



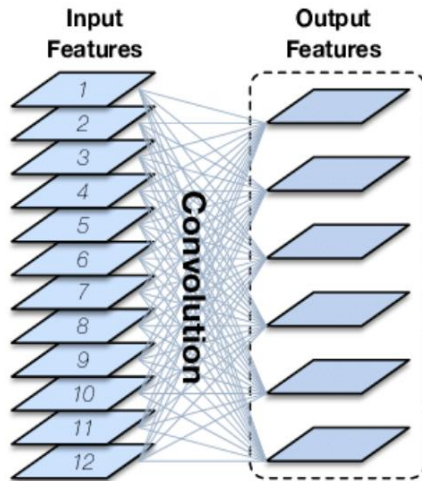
b)

Dilation=2
(卷积核 3×3)



c)

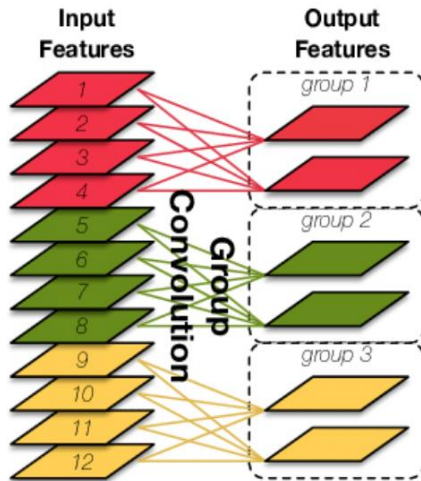
Dilation=3
(卷积核 3×3)



标准卷积

卷积核 $3 \times 3 \times 12$

参数量: $6 \times 3 \times 3 \times 12 + 6$



组卷积

组数为3, 卷积核 $3 \times 3 \times 4$

参数量: $3 \times 2 \times 3 \times 3 \times 4 + 6$

Group Convolution

对于pytorch深度学习框架，**二维卷积nn.Conv2d**对于高度 H 和宽度 W 都进行卷积运算（ $[N, \mathbf{C}_{in}, H_{in}, W_{in}] \rightarrow [N, \mathbf{C}_{out}, H_{out}, W_{out}]$ ）

```
class torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0, dilation=1, groups=1, bias=True)
```

in_channels(int) 输入数据体的通道数 $\mathbf{C}_{in}$, **out_channels(int)** 输出数据体的通道数 $\mathbf{C}_{out}$

kernel_size(int or tuple) 卷积核大小，可以是int，或tuple；kernel_size=2，意味着卷积大小(2,2)，kernel_size=(2,3)，意味着卷积大小(2, 3)即非正方形卷积

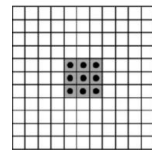
stride(int or tuple, optional) 卷积核的步长，默认为1，与kernel_size类似，stride=2,意味着步长上下左右扫描皆为2，stride=(2,3)，左右扫描步长为2，上下为3

padding(int or tuple, optional) 输入的每一条边补充0的层数，默认为0

dilation(int or tuple, optional) 卷积核元素间的距离，默认为1

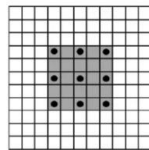
groups(int, optional) 将原始输入通道划分成的组数，默认为1

bias(bool, optional) 默认为True，表示输出的bias可学习



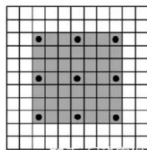
a)

Dilation=1



b)

Dilation=2



c)

Dilation=3

在标准的二维卷积操作中，滤波器（卷积核）与输入数据体之间必须满足什么重要原则？

- ☐ A 滤波器和输入数据体必须具有相同通道数
- ☐ B 滤波器的尺寸必须与输入数据体的尺寸相同
- ☐ C 滤波器的步长必须为1
- ☐ D 输入数据体必须是单通道

提交

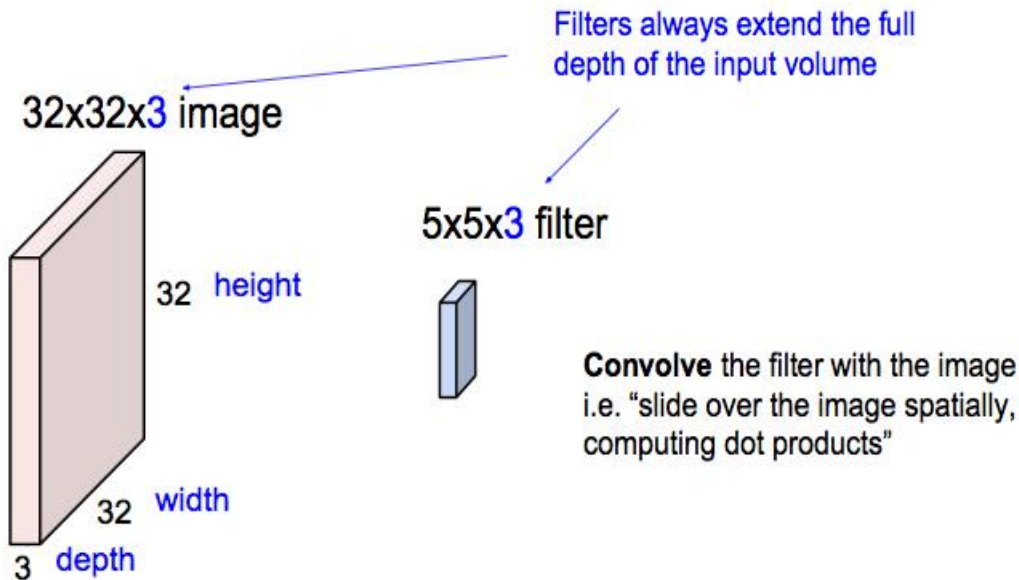


在标准的二维卷积操作中，滤波器（卷积核）与输入数据体之间必须满足什么重要原则？

- ☒ A 滤波器和输入数据体必须具有相同通道数
- ☐ B 滤波器的尺寸必须与输入数据体的尺寸相同
- ☐ C 滤波器的步长必须为1
- ☐ D 输入数据体必须是单通道

提交

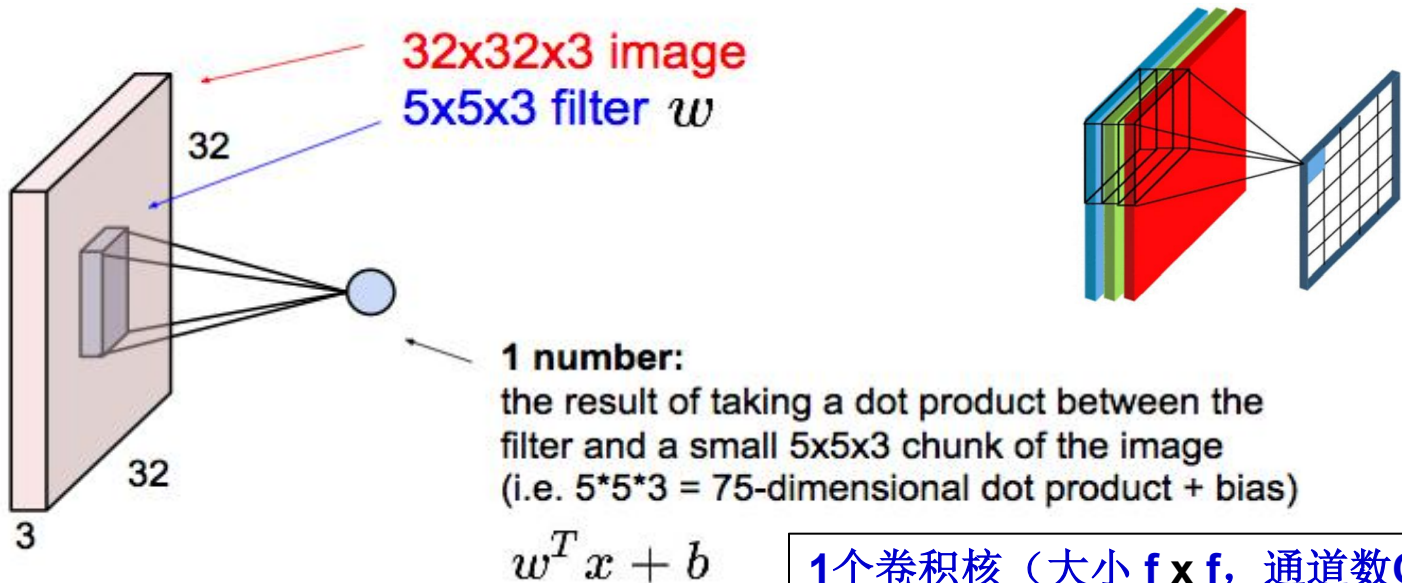




一个重要的原则：
滤波器的通道数和输入的通道相同，输出的特征通道数则跟滤波器数量相关

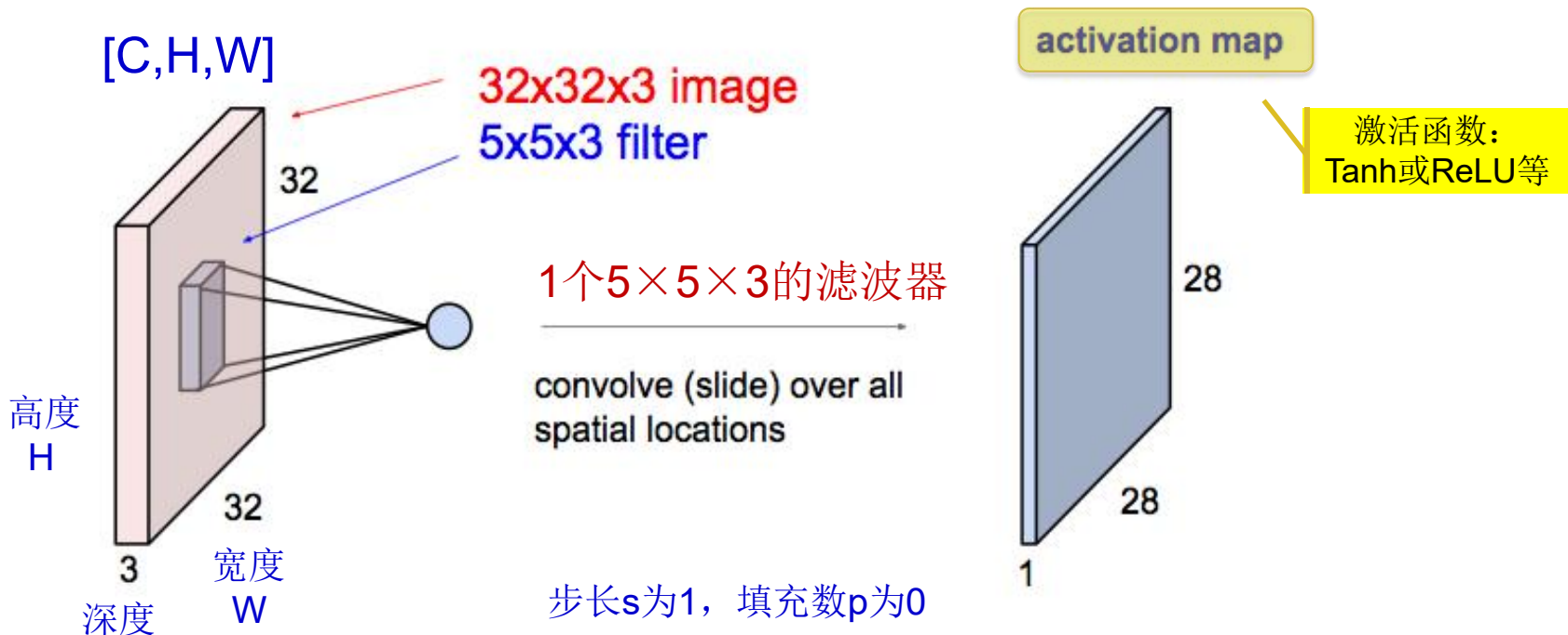
输入图像: 3阶张量 (tensor) [C,H,W]

height和width表示图像的分辨率H x W， depth表示通道数C， 像素值为0-255之间的整数。



1个卷积核（大小 $f \times f$ ，通道数 C ，有偏置）的参数量： $C \times f \times f + 1$

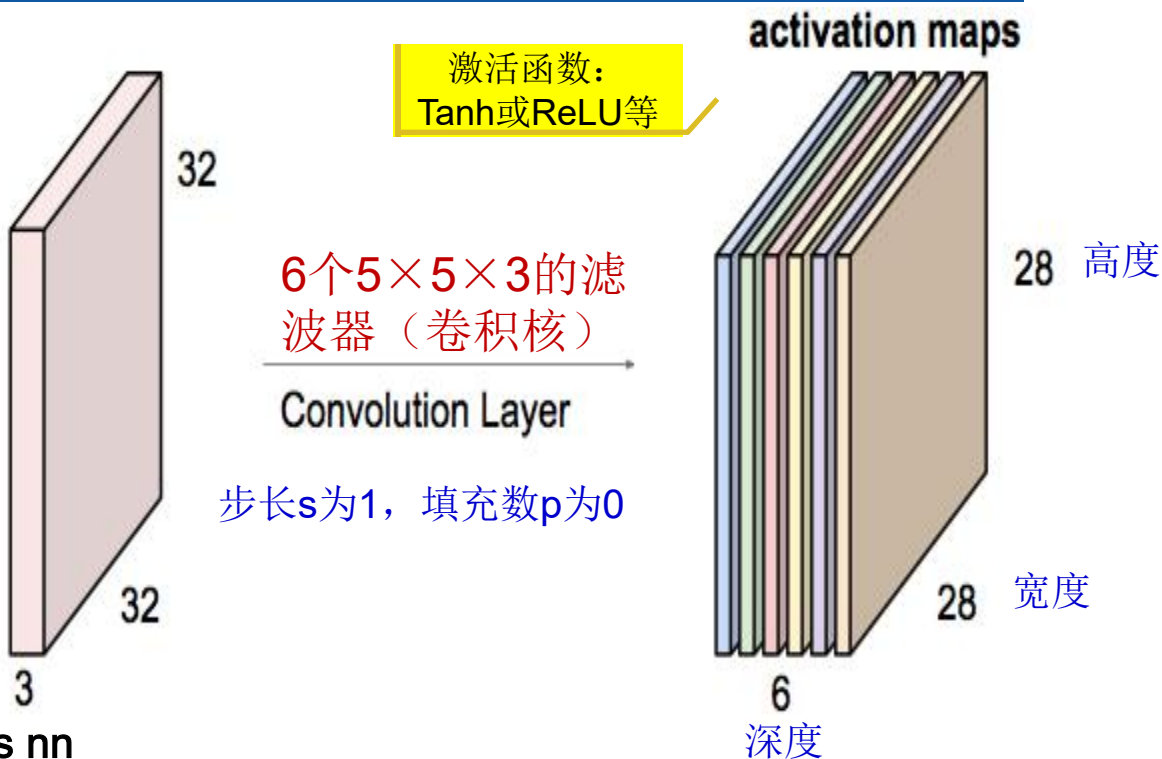
卷积（1个卷积核，大小 $f \times f$ ，通道数 C ，有偏置）1次的计算量：
 通道数 $\times$ （每个通道的乘法次数+加法次数）+通道相加次数+偏置加法次数
 $= C \times (f \times f + f \times f - 1) + C - 1 + 1 = 2 \times C \times f \times f$



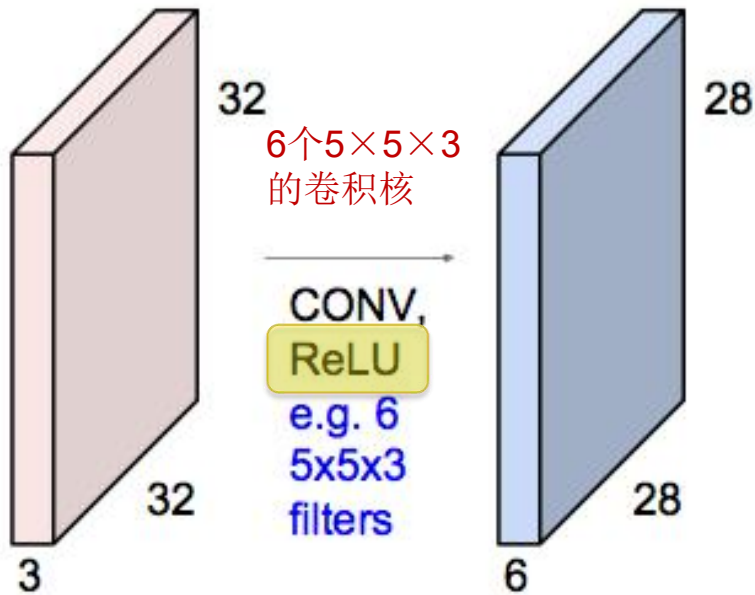
输出特征图的宽度/高度: $(W/H + 2p - f) / s + 1 = (32 + 2 \times 0 - 5) / 1 + 1 = 28$

1个卷积核 (默认有偏置) 的参数量: $3 \times 5 \times 5 + 1 = 76$

输出特征图的计算量: $1 \times 28 \times 28 \times 2 \times 3 \times 5 \times 5 = 117600$



```
import torch.nn as nn
conv_layer = nn.Sequential(
    nn.Conv2d(in_channels=3, out_channels=6, kernel_size=5, stride=1, padding=0),
    nn.ReLU() )
```



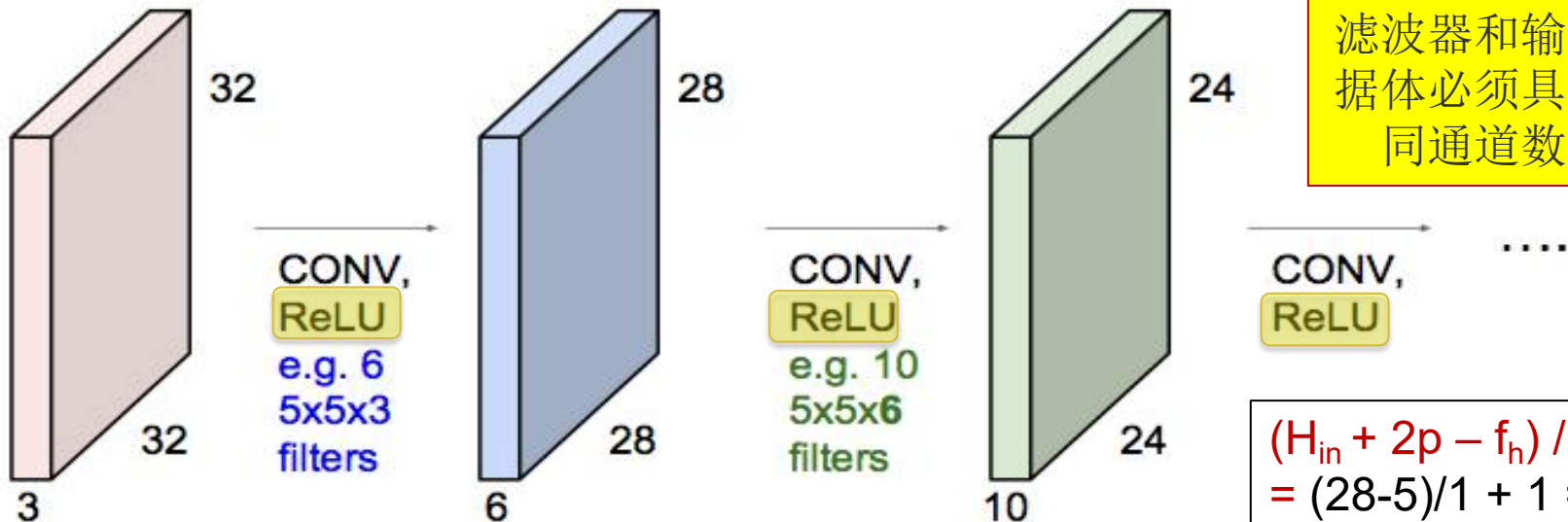
多卷积核：不同卷积核提取不同特征

- 为了从图像中提取多种形式的特征，通常使用多个滤波器对输入图像进行不同的卷积操作
- 为了提取不同抽象级别的特征，通常采用多个卷积层

stride为1，没有padding

已知输入图像尺寸为 $6 \times 28 \times 28$ ，卷积层使用10个卷积核，其中卷积核大小为 5×5 ，步长为1，无填充，则卷积核的通道数为 [填空1]，输出特征图的个数为 [填空2]，高度为 [填空3]，宽度为 [填空4]。

卷积：步长s为1，填充数p为0

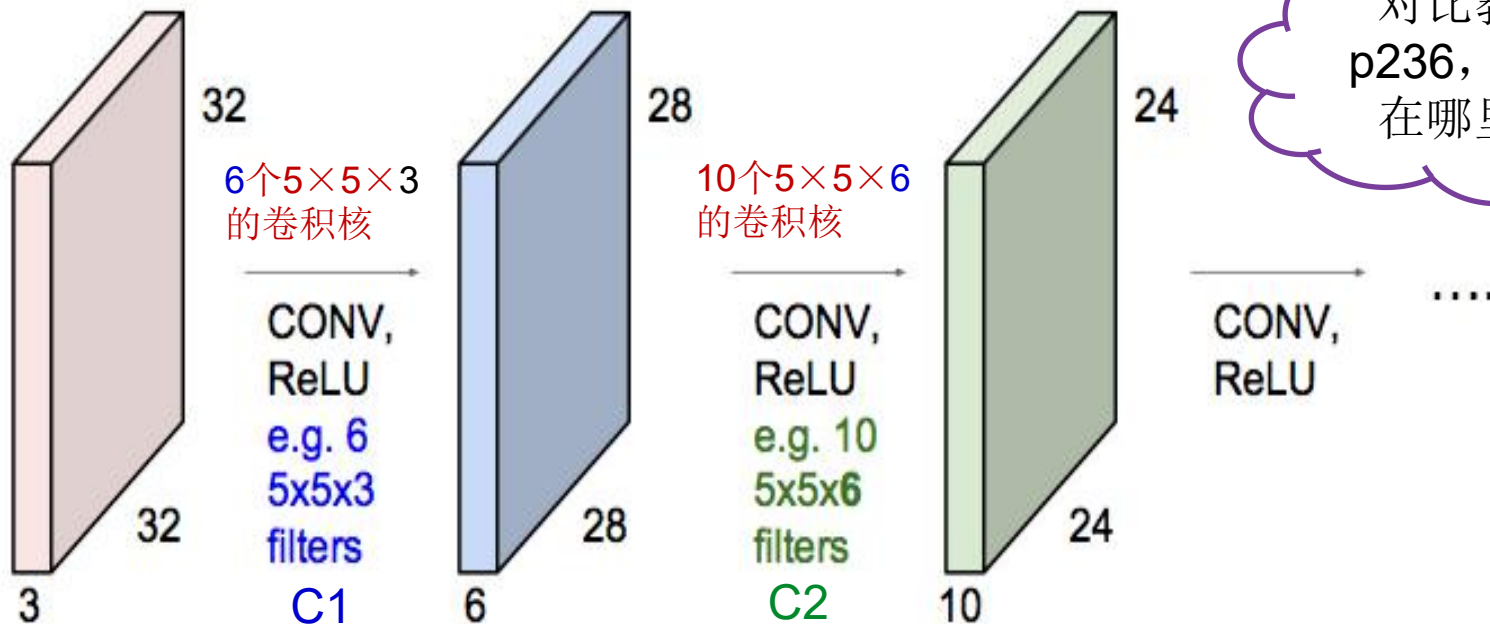


一个重要的原则：
滤波器和输入数
据体必须具有相
同通道数。

$$(H_{in} + 2p - f_h) / s + 1 = (28-5)/1 + 1 = 24$$

```
import torch.nn as nn
conv_layer = nn.Sequential(
    nn.Conv2d(in_channels=3, out_channels=6, kernel_size=5, stride=1, padding=0),
    nn.ReLU(),
    nn.Conv2d(in_channels=6, out_channels=10, kernel_size=5, stride=1, padding=0),
    nn.ReLU() )
```

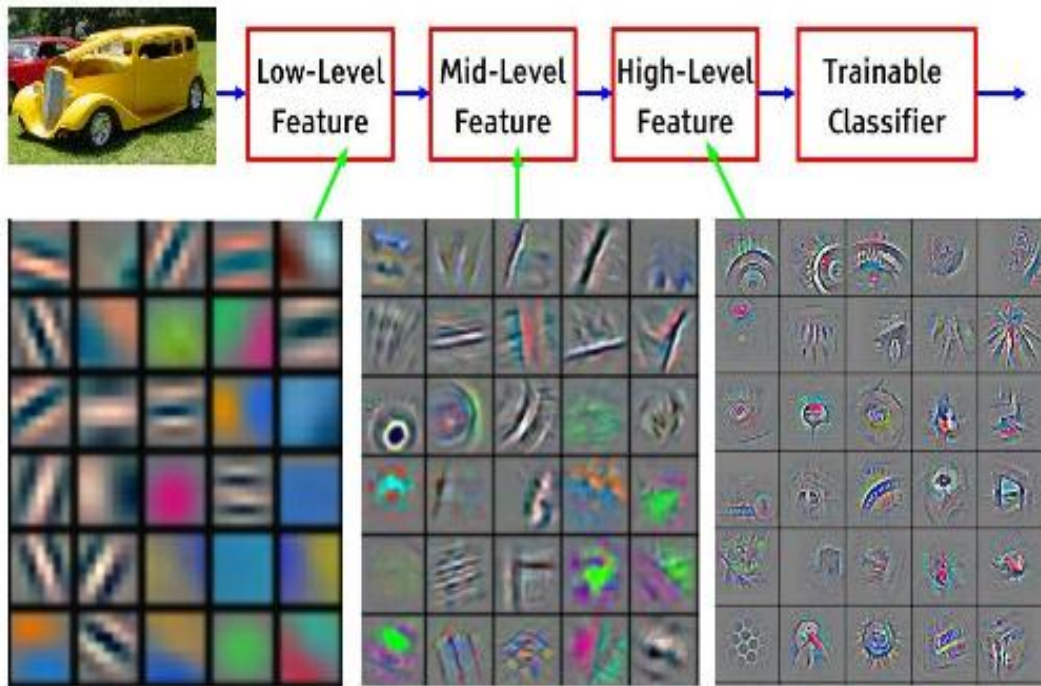
stride为1, 没有padding



第1个卷积-激活层C1, 有6个特征图, 特征图的大小为 28×28 。特征图中每个神经元与输入图像中 5×5 的邻域 (感受野) 相连。C1有 $6 \times (5 \times 5 \times 3 + 1) = 456$ 个训练参数, 共 $6 \times (28 \times 28) \times (5 \times 5 \times 3 + 1) = 357,504$ 个连接。

Preview

[From recent Yann
LeCun slides]



Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]

2013年，Matthew D. Zeiler 等人，通过将每层的特征图利用“**转置卷积**”网络映射回输入图片，即可查看学到的特征分布

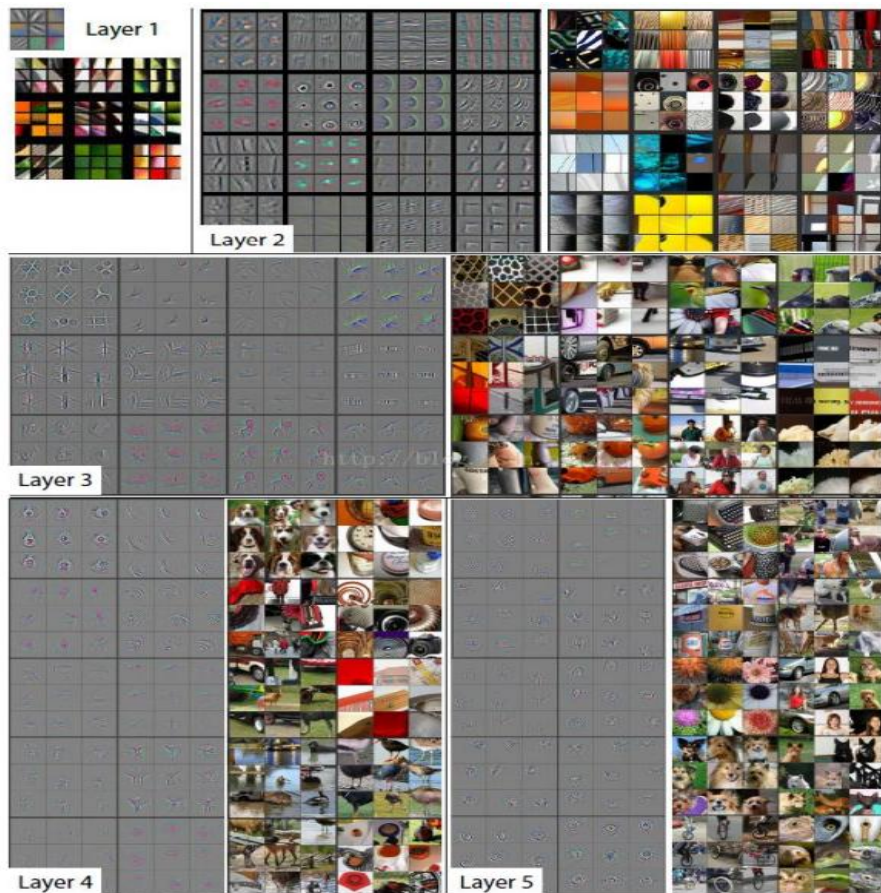
转置卷积:能把小特征图放大，用于图像生成、语义分割

第二层的特征对应到**边、角、色彩**等底层图像提取；

第三层开始捕获到**纹理**这些中层特征；

第四、五层呈现了**物体的部分特征**，如小狗的脸部、鸟类的脚部等高层特征。

Zeiler M D, Fergus R. Visualizing and Understanding Convolutional Networks . ECCV2014, 8689:818-833

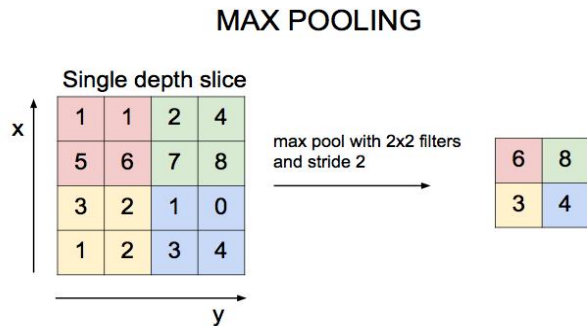


- 一般情况下，在连续的卷积层之间会周期性地插入一个池化层（也称**汇聚层**）
- 池化层作用：高效降维，提升鲁棒性、减少参数量和计算量，避免过拟合
- 最大化池化（**Max Pooling**）：取局部区域的最大值，保留局部最显著的特征（如边缘、纹理的强度峰值），提供平移不变性（对输入的微小变化不敏感）

`torch.nn.MaxPool1d`, `torch.nn.MaxPool2d`, `torch.nn.MaxPool3d`

- 平均池化（**Average/Mean Pooling**）和 L-2 范式池化（**L2-norm Pooling**）

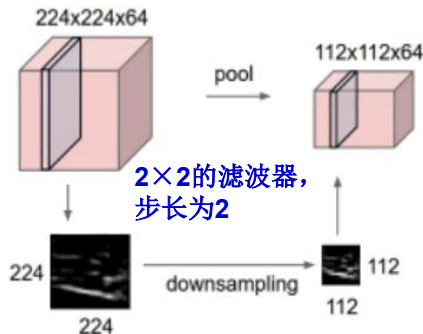
`torch.nn.AvgPool1d`, `torch.nn.AvgPool2d`, `torch.nn.AvgPool3d`



输入数据体 $H_{in} * W_{in}$ ，滤波器 $f_h * f_w$ ，步长 s 、零填充数 p ，则输出数据体：

$$\left\lfloor \frac{(H_{in} + 2p - f_h)}{s} + 1 \right\rfloor$$

$$\left\lfloor \frac{(W_{in} + 2p - f_w)}{s} + 1 \right\rfloor$$



```
import torch.nn as nn
pool_layer = nn.MaxPool2d(kernel_size=2, stride=2, padding=0)
```

教材p237: LeNet-5池化运算，池化核大小为 2×2 ，采样方式为4个输入相加，乘以一个可训练参数，再加上一个可训练偏置，结果最后通过sigmoid，所以下采样的参数个数是 $(1 + 1) \times 6$ 而不是零。

关于池化层的作用，以下哪一项描述是错误的？

- ☐ A 池化层可以减少模型的参数量和计算量
- ☐ B 池化层能够提升特征的空间分辨率
- ☐ C 池化层有助于提高模型对输入微小变化的鲁棒性
- ☐ D 池化层可用于防止过拟合

提交



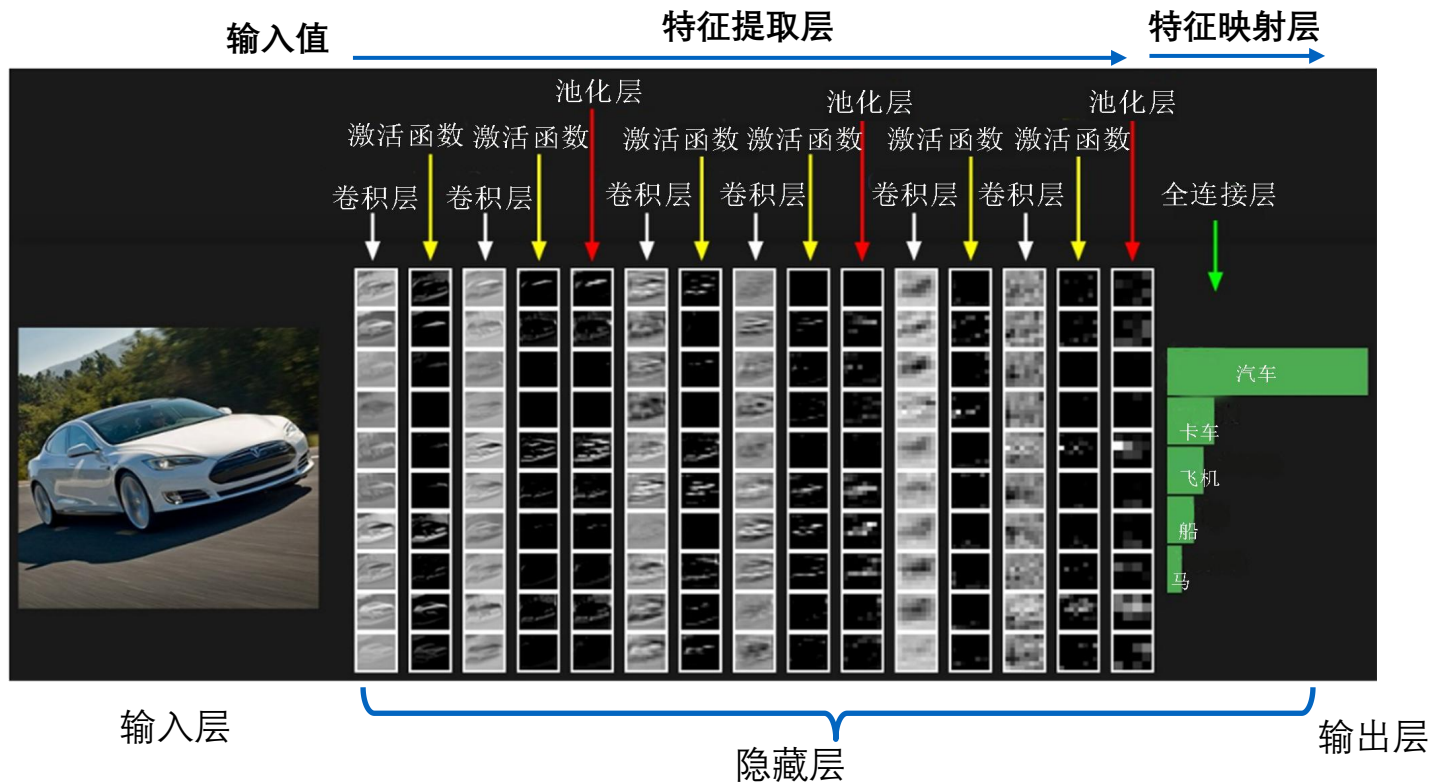
关于池化层的作用，以下哪一项描述是错误的？

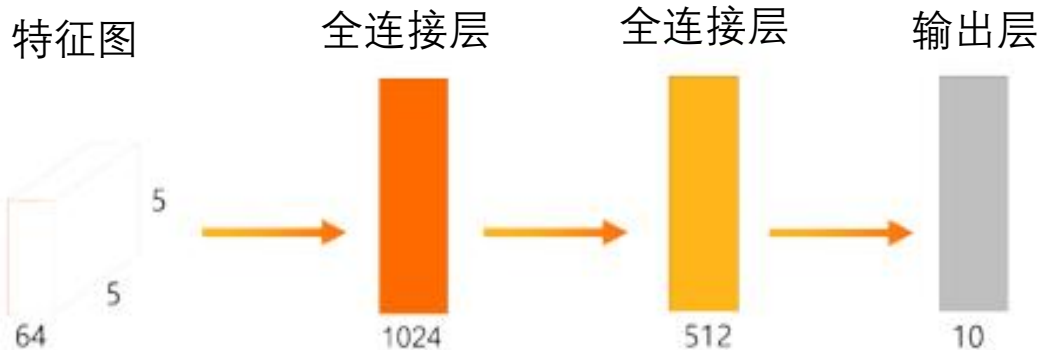
- ☐ A 池化层可以减少模型的参数量和计算量
- ☒ B 池化层能够提升特征的空间分辨率
- ☐ C 池化层有助于提高模型对输入微小变化的鲁棒性
- ☐ D 池化层可用于防止过拟合

提交

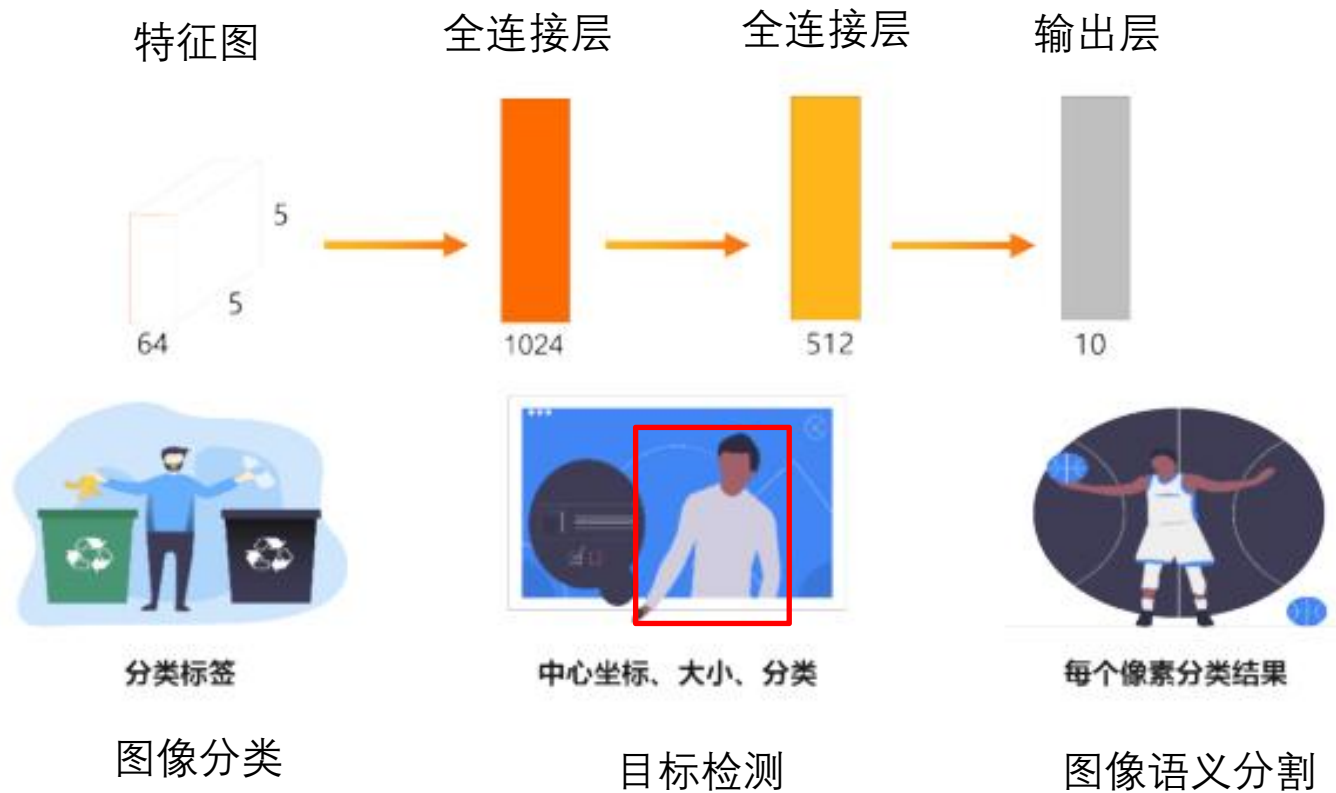


全连接层（Fully Connected Layer，简称 Linear 层）负责将提取到的特征进行全局整合并映射到输出空间（如分类任务的类别概率、回归任务的预测值）。



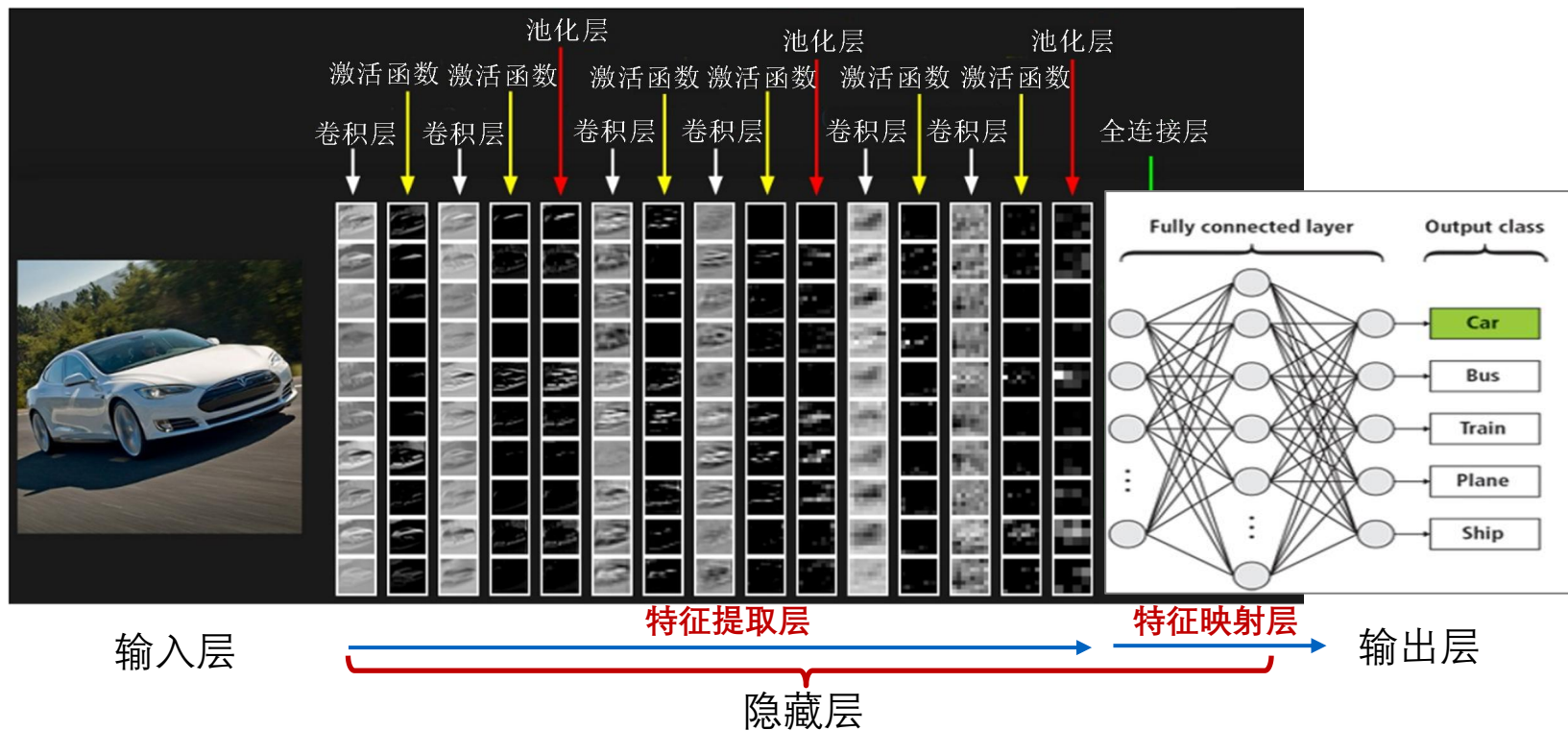


```
import torch.nn as nn
fc_layers = nn.Sequential(
    nn.Linear(in_features=64 * 5 * 5, out_features=1024), # 全连接层1: 输入特征数64*5*5, 输出1024
    nn.ReLU(),
    nn.Linear(in_features=1024, out_features=512), # 全连接层2: 输入1024, 输出512
    nn.ReLU(),
    nn.Linear(in_features=512, out_features=10) ) # 输出层用softmax激活（多分类单标签任务）
```

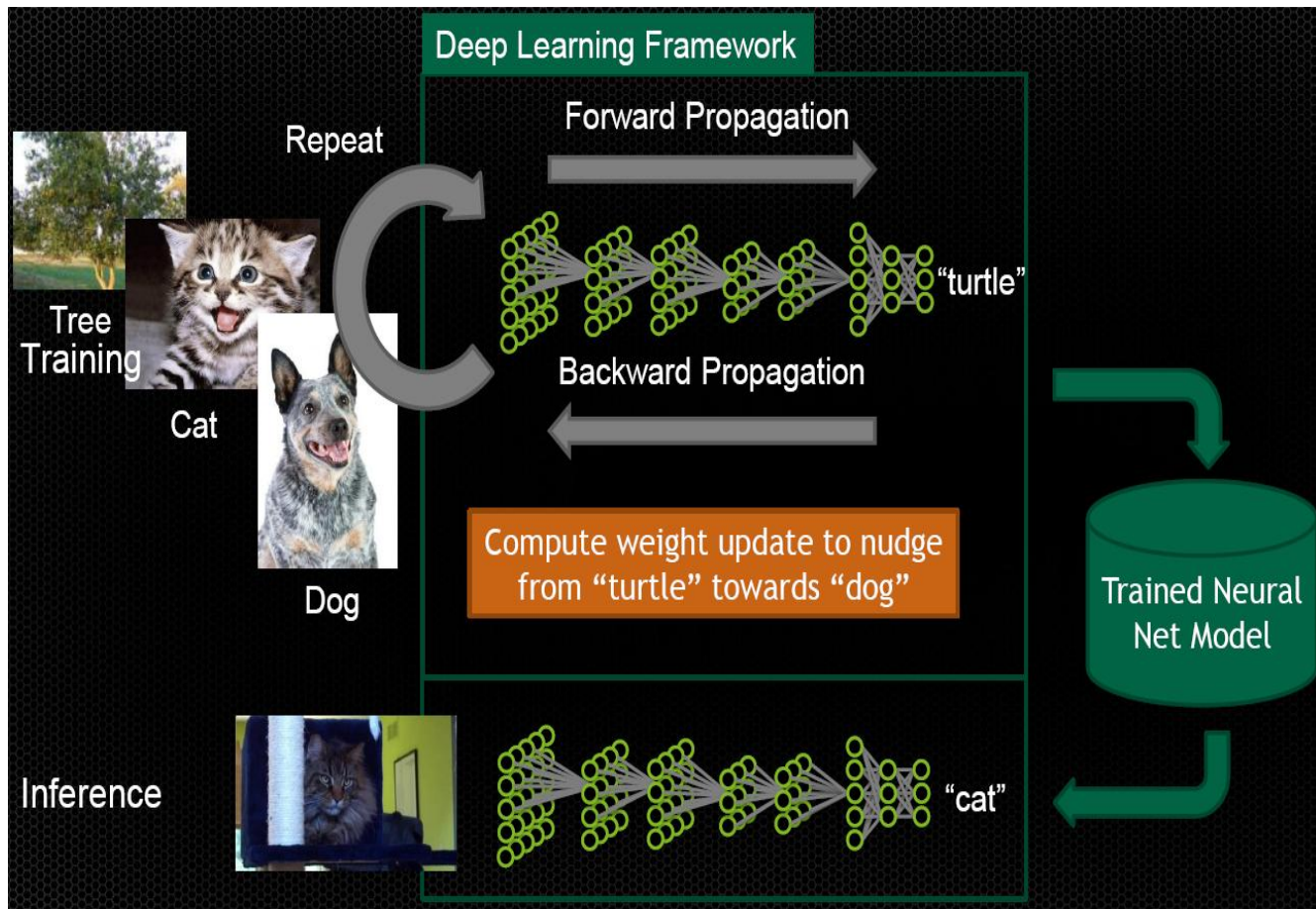


一个典型的卷积神经网络一般由卷积层、池化层、全连接层堆叠而成。

卷积神经网络的结构特性：局部连接，权重共享、空间或时间上的下采样。



卷积神经网络的关键技术：
局部连接、参数（权值）共享、多卷积核、池化

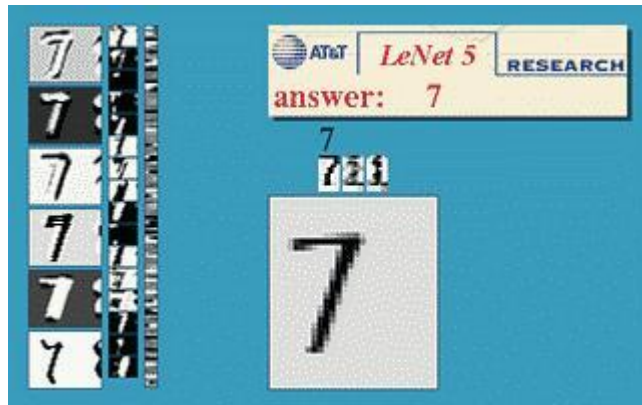
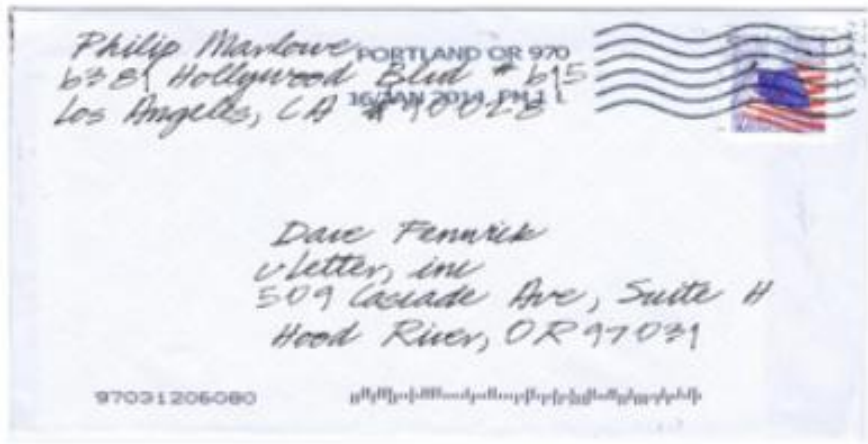




[LeCun et al., 1998]

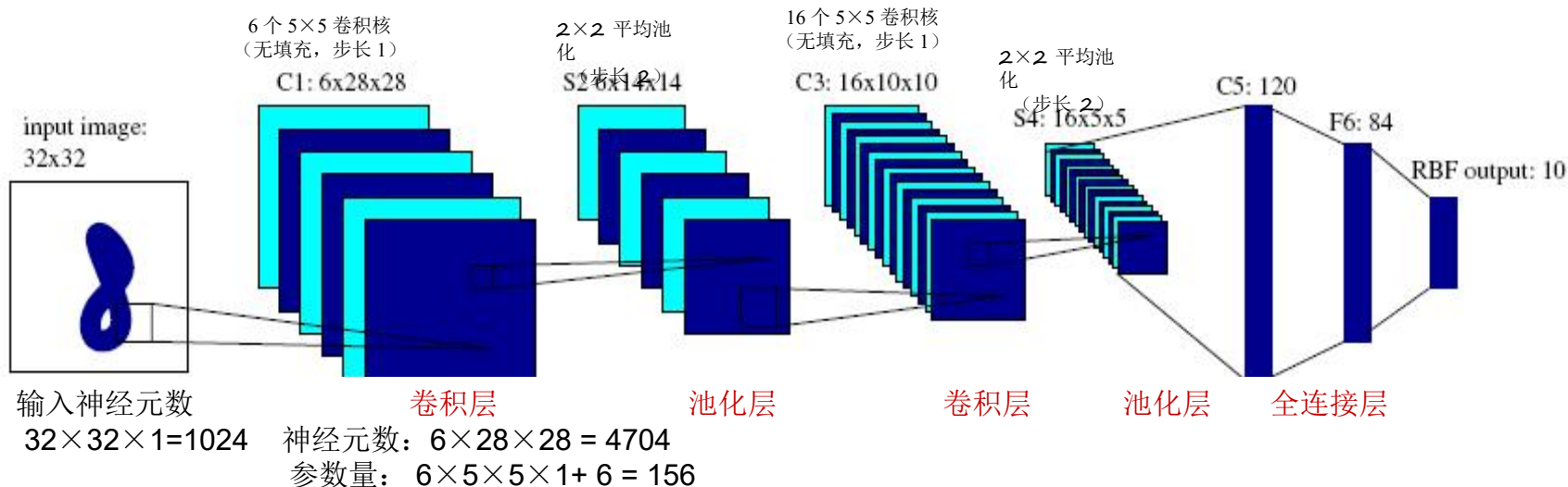
LeNet-5是由Yann LeCun等人在1998年提出的一种用于手写数字识别的卷积神经网络，是早期卷积神经网络中最有代表性的模型之一。

在 20 世纪 90 年代时，基于 LeNet-5 的手写数字识别系统被美国很多银行使用，用来识别支票上面的手写数字



[Yann LeCun](#), [Léon Bottou](#), [Yoshua Bengio](#), [Patrick Haffner](#): Gradient-based learning applied to document recognition. [Proc. IEEE 86\(11\)](#): 2278-2324 (1998)

LeNet-5主要有2个卷积层、2个下采样层（池化层）、3个全连接层，网络结构如下



- 卷积: 使用卷积提取空间特征, 层与层之间的稀疏连接矩阵避免了高额的计算成本
- 池化: 使用映射到空间均值下采样 (Subsample)
- 非线性: 使用双曲正切 (tanh) 或S型 (Sigmoid) 形式的非线性
- 分类: 使用多层神经网络 (MLP) 作为最后的分类器

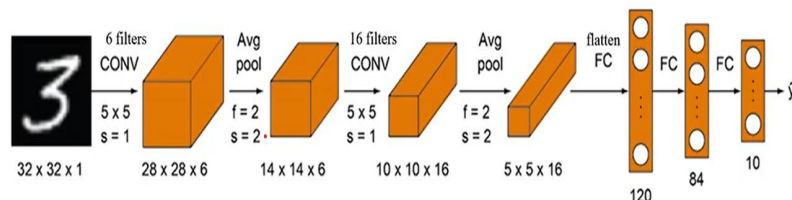
教材p237图
8.15

```

import torch.nn as nn
class LeNet5(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(1, 6, 5, padding=0), # 1*32*32 → 6*28*28
            nn.Tanh(),
            nn.AvgPool2d(2, 2),           # 6*28*28 → 6*14*14
            nn.Conv2d(6, 16, 5, padding=0), # 6*14*14 → 16*10*10
            nn.Tanh(),
            nn.AvgPool2d(2, 2)           # 16*10*10 → 16*5*5
        )
        self.fc = nn.Sequential(
            nn.Flatten(),
            nn.Linear(16*5*5, 120),
            nn.ReLU(),
            nn.Linear(120, 84),
            nn.ReLU(),
            nn.Linear(84, 10)
        )
    def forward(self, x):
        return self.fc(self.conv(x))
model = LeNet5().to(device)
print(model)

```

如果输入图像
32*32*1改为
28*28*1, 怎
么调整模型?



池化层的输入输出通道数相同

```

LeNet5(
  (conv): Sequential(
    (0): Conv2d(1, 6, kernel_size=(5, 5), stride=(1, 1))
    (1): Tanh()
    (2): AvgPool2d(kernel_size=2, stride=2, padding=0)
    (3): Conv2d(6, 16, kernel_size=(5, 5), stride=(1, 1))
    (4): Tanh()
    (5): AvgPool2d(kernel_size=2, stride=2, padding=0)
  )
  (fc): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=400, out_features=120, bias=True)
    (2): ReLU()
    (3): Linear(in_features=128, out_features=84, bias=True)
    (4): ReLU()
    (5): Linear(in_features=84, out_features=10, bias=True)
  )
)

```

nn.Conv2d()、nn.AvgPool2d()
输入: [N, Cin, Hin, Win]
输出: [N, Cout, Hout, Wout]

nn.Linear()
输入: [N, in_features]
输出: [N, out_features]

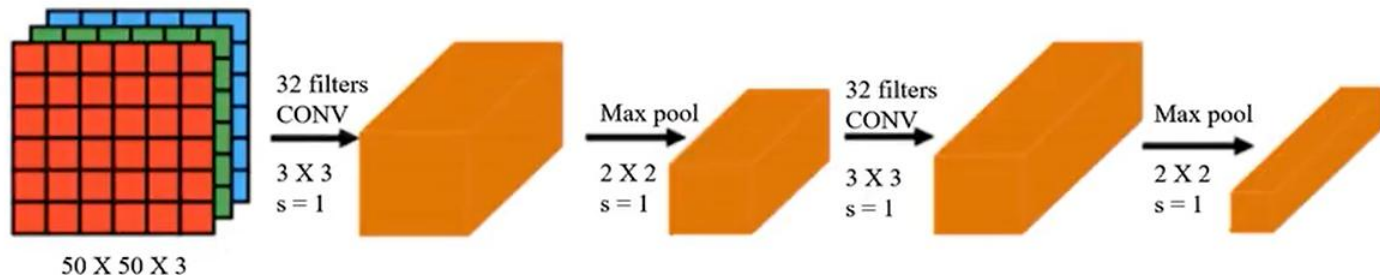


本实例的主要任务是：

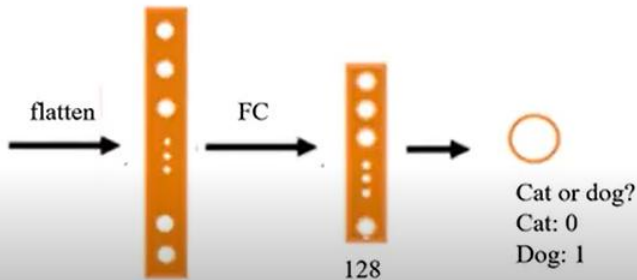
- 1) 基于dataset\training_set数据（2.5万猫狗图片），根据提供的结构，建立CNN模型：
- 2) 识别图片中的猫\狗、计算dataset\test_set（2千猫狗图片）测试数据预测准确率
- 3) 下载猫\狗图片，对其进行预测。

模型设计

CNN结构:



备注：卷积层采用relu作为输出时的激活函数



1.数据预处理+加载

```
from torchvision.datasets import ImageFolder
transform = transforms.Compose([transforms.Resize((50,50)),
transforms.ToTensor()]) # 归一化+转Tensor
train_ds = ImageFolder('./data/train_set', transform=transform)
test_ds = ImageFolder('./data/test_set', transform=transform)
train_loader = DataLoader(train_ds, 32, shuffle=True, num_workers=0)
test_loader = DataLoader(test_ds, 32, shuffle=False, num_workers=0)

print(f'数据加载成功： 训练{len(train_ds)}张， 测试{len(test_ds)}张 |
类别映射: {train_ds.class_to_idx}')
```

2.建立CNN模型

```
class BinaryCNN(nn.Module):  
    def __init__(self):  
        super().__init__()  
        self.cnn = nn.Sequential(  
            nn.Conv2d(3,32,3), nn.ReLU(), nn.MaxPool2d(2,2),  
            nn.Conv2d(32,32,3), nn.ReLU(), nn.MaxPool2d(2,2),  
            nn.Flatten(),  
            nn.Linear(32*11*11, 128), nn.ReLU(),  
            nn.Linear(128,1), nn.Sigmoid()  
        )  
    def forward(self, x): return self.cnn(x)
```

3.模型优化配置

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')  
model = BinaryCNN().to(device)  
criterion = nn.BCELoss()  
optimizer = optim.Adam(model.parameters(), lr=1e-3)  
print(f'使用设备: {device}')
```

4.训练模型

```
print(f"\n训练{epochs}轮...")
for epoch in range(epochs):
    model.train()
    trn_l, trn_a, trn_t = 0,0,0
    for imgs, labs in train_loader:
        imgs, labs = imgs.to(device), labs.to(device).float().unsqueeze(1)
        out = model(imgs)
        loss = criterion(out, labs)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        trn_l += loss.item()*imgs.size(0)
        trn_a += (out>=0.5).eq(labs).sum().item()
        trn_t += imgs.size(0)
```

5. 模型评估

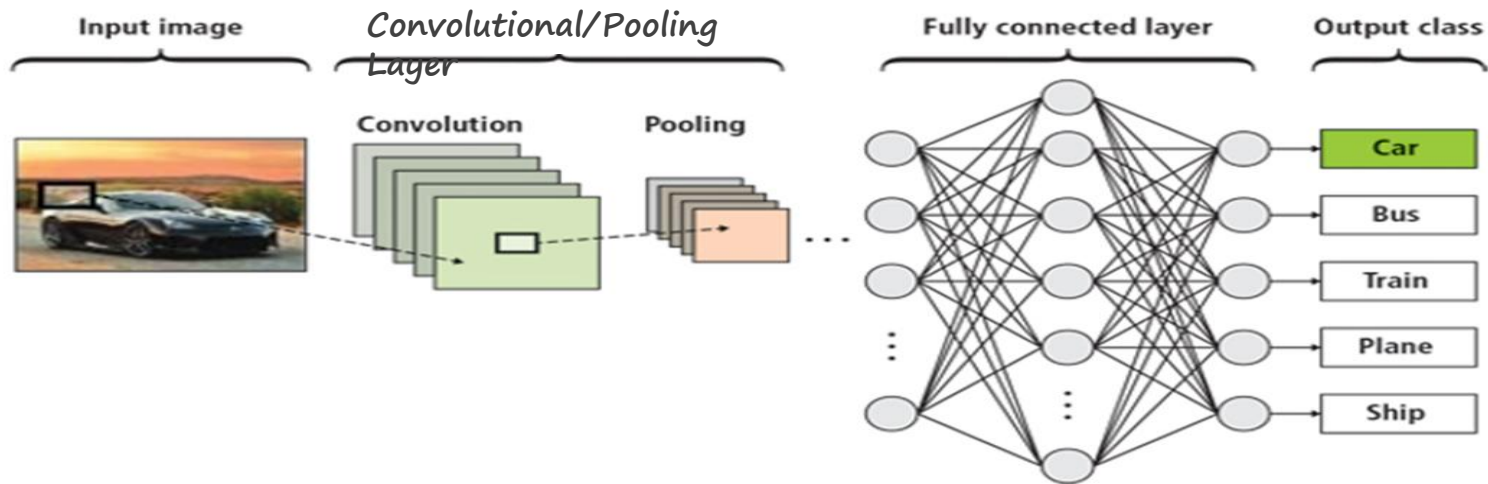
```
model.eval()
tst_l, tst_a, tst_t = 0,0,0
with torch.no_grad():
    for imgs, labs in test_loader:
        imgs, labs = imgs.to(device), labs.to(device).float().unsqueeze(1)
        out = model(imgs)
        tst_l += criterion(out, labs).item()*imgs.size(0)
        tst_a += (out>=0.5).eq(labs).sum().item()
        tst_t += imgs.size(0)
```

6. 实例测试

```
img_path = './data/test.jpg'
img = transform(Image.open(img_path).convert('RGB')).unsqueeze(0).to(device)
model.eval()
    with torch.no_grad():
        prob = model(img).item()
    print(f"\n预测： {'类别1' if prob>=0.5 else '类别0'} （概率： {prob:.4f} ） ")
```

核心组件：

- 卷积层（*Convolutional Layer*）：使用滑动窗口（卷积核）提取局部特征，共享参数。
- 池化层（*Pooling Layer*）（如最大池化）：降维、增强平移不变性。
- 全连接层：将提取到的特征进行整合、维度变换并输出预测结果。



假设用卷积神经网络(1个卷积层、1个池化层和1个全连接层)实现图像分类(有3个类别)，输入为 $16 \times 16 \times 3$ 的彩色图像(其中3为通道数)，其卷积层采用10个卷积核(滤波器)和ReLU激活函数，卷积核的大小为 5×5 ，步长为1，用1层零填充，无偏置；池化层采用 2×2 的滤波器，步长为2的最大池化，无填充；全连接层无偏置。问：

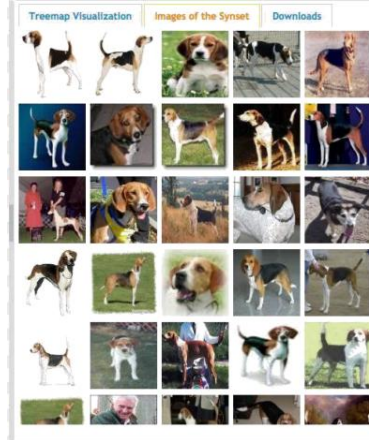
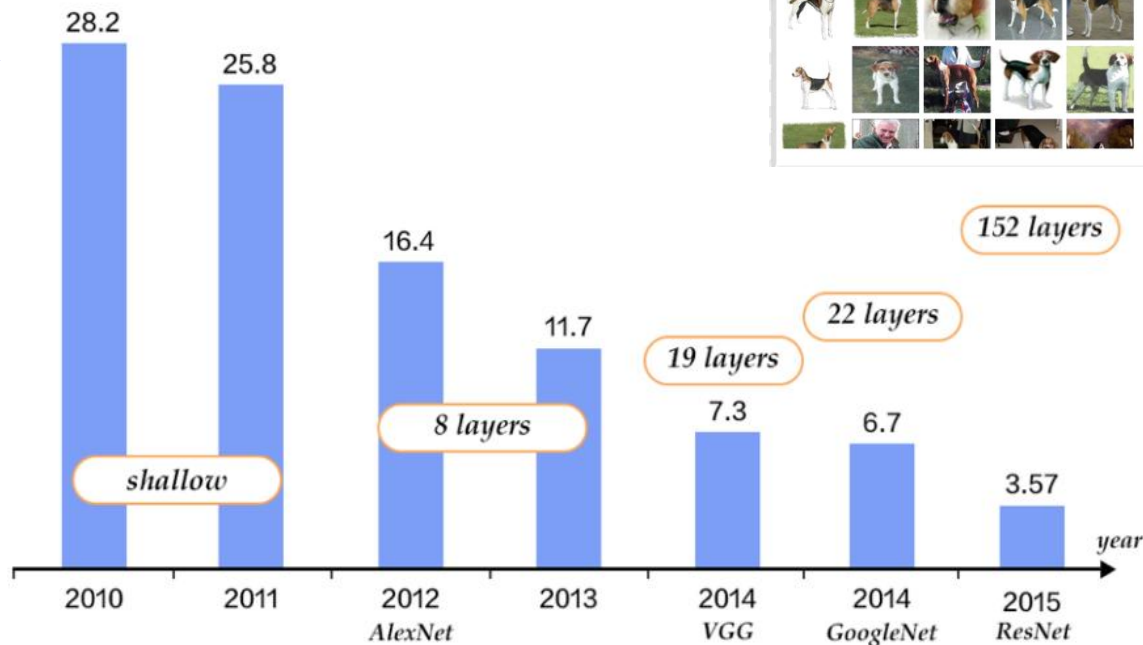
- 1) 卷积层输出为多少个大小为多少的特征图，有多少个训练参数，共多少个连接？卷积层的计算量是多少？
- 2) 池化层输出大小为多少的特征图，共有多少个神经元？
- 3) 全连接层有多少个神经元，多少个训练参数？全连接层的计算量是多少？

ImageNet: 超过1400万的图像, 包含2万多个类别

• ImageNet ILSVRC 1000 个类别

(ImageNet [Large Scale Visual Recognition Challenge](#))

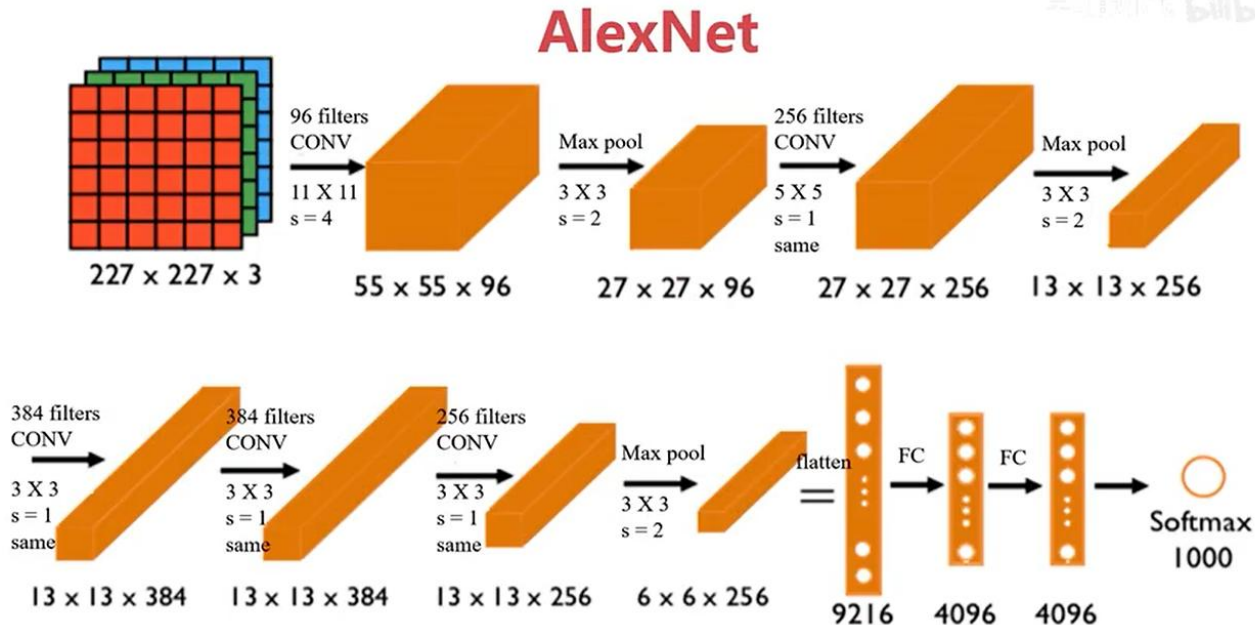
Top-5错误率



ImageNet 1K 或者ILSVRC2012: 有1000类, 每类大约有1000张图像, 有大约120多万的训练集 (150G), 5万验证集, 10万测试集。

AlexNet是2012年由Alex Krizhevsky等人提出的。

- 获得2012年ImageNet图像分类大赛冠军：错误率25.8% -> 16.4%
- 网络更深：8层



AlexNet的主要特点

- 使用 $ReLU$ 作为非线性激活函数
- 在训练的时候使用 $Dropout$ 随机地丢弃神经元，避免模型过拟合
- 使用效果更好的有重叠的最大池化代替避免平均池化
- 使用数据增强的方式增加训练样本
- 设计了 LRN ($Local Response Normalization$) 层，利用邻近的数据做归一化
- 使用多 GPU 并行计算，大幅度减少了训练时间



数据增强



不同大小的原图缩放到固定大小



CGAN 网络生成的手写数字图片



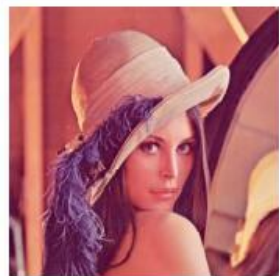
旋转图片



水平翻转



竖直翻转



裁剪前



裁剪并缩放后-1



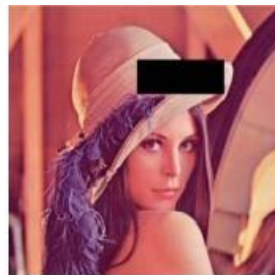
裁剪并缩放后-2



添加高斯噪声



变换视角



随机擦除

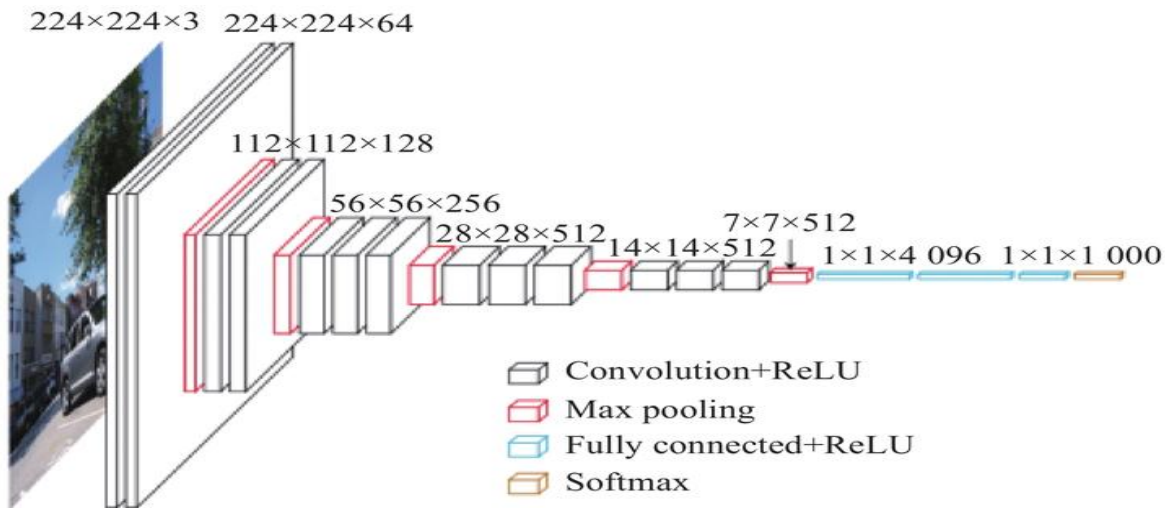


8.3

典型的卷积神经网络-VGG

VGG 由牛津大学 Visual Geometry Group (VGG) 提出

- 获得2014年ILSVRC挑战的亚军：11.7% $\rightarrow$ 7.3%
- **VGG16**: 包含16个带权重的层（13个卷积层和3个全连接层）
- 依次采用 **多个3*3卷积**，达到更大的感受野类似的效果



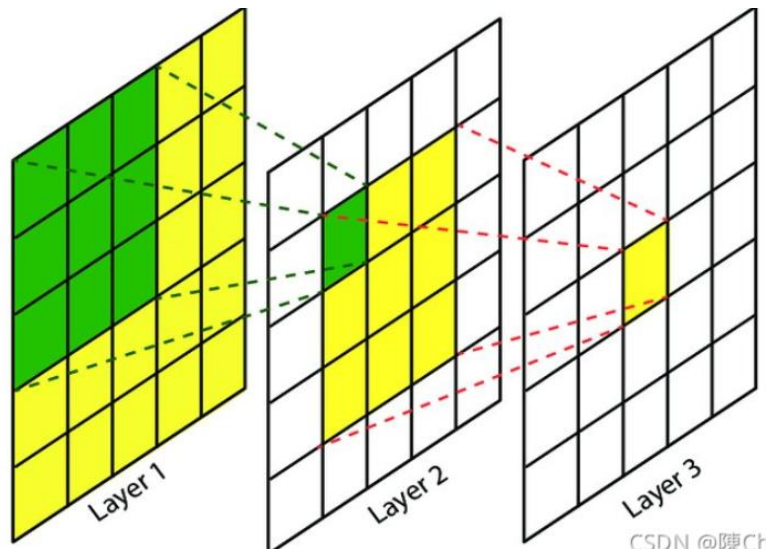
VGG16

VGG中，使用3个 3×3 卷积核代替 7×7 卷积核，使用2个 3×3 卷积核代替 5×5 卷积核，主要目的：

- 1) 具有相同感受野的条件下，增加网络的深度，提升神经网络效果；
- 2) 减少网络的参数量。

2个 3×3 的卷积核堆叠获得的感受野相当于一个 5×5 的卷积核

3个 3×3 的卷积核堆叠获得的感受野相当于一个 7×7 的卷积核



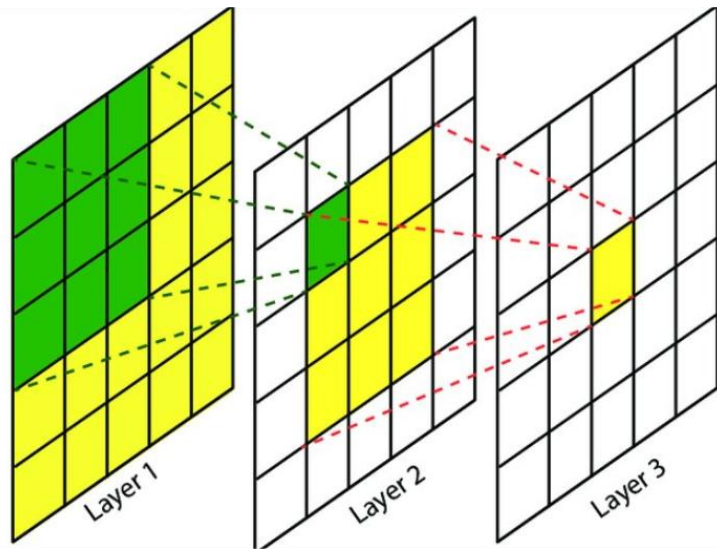
① 2个3x3卷积层和1个5x5卷积层感受野相同

假设图片尺寸为 28×28 ，使用 5×5 卷积核， $\text{stride}=1$ ，得到特征图尺寸：

$$(28-5)/1+1=24$$

使用2个 3×3 卷积核（这里2个是指2层）， $\text{stride}=1$ ：

第一个 3×3 卷积层后： $(28-3)/1+1=26$ ；第
二个 3×3 卷积层后： $(26-3)/1+1=24$



② 减少网络参数：2个 3×3 卷积层，所用的参数总量： $2 \times (3 \times 3) \times \text{channels}$
（假设无偏置） 1个 5×5 卷积层，所用的参数总量： $5 \times 5 \times \text{channels}$

关于VGG网络结构的优化，下列说法正确的是？
(多选)

- ☒ A 使用多个 3×3 卷积核堆叠可以增加网络深度
- ☒ B 多个小卷积核堆叠可以减少参数量
- ☒ C 3个 3×3 卷积核堆叠的感受野等同于一个 7×7 卷积核
- ☒ D 2个 3×3 卷积核堆叠的感受野等同于一个 5×5 卷积核

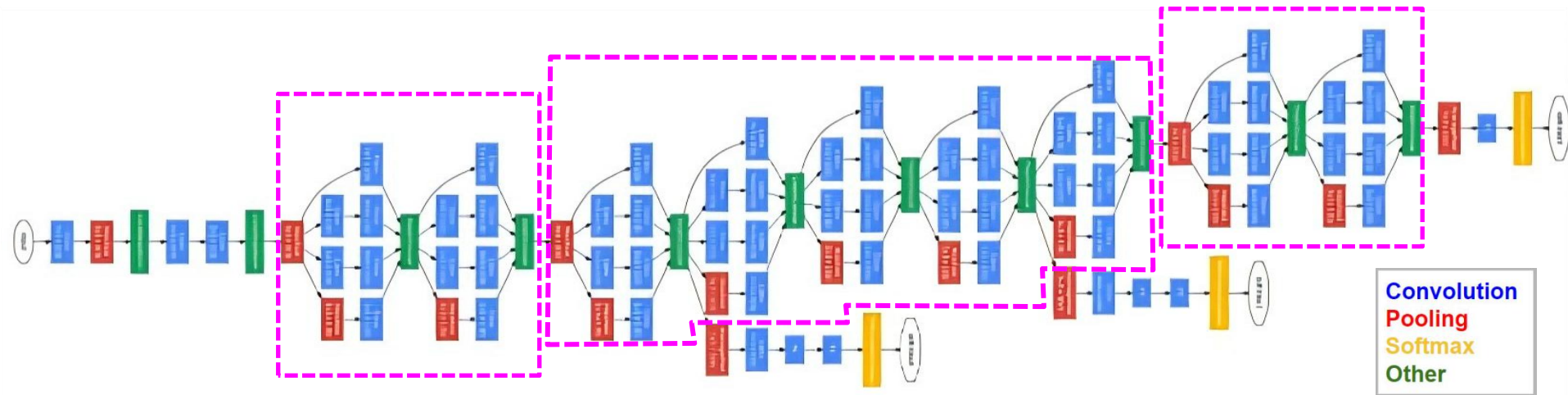
提交



- Christian Szegedy等（Google Research）提出，由 22 层网络构成
- 没有最深，只有更深
- 获得 2014 年 ImageNet 挑战的冠军：6.7%
- 采用 NIN（Network in Network）模型和 Inception 模块

Inception结构：将不同的卷积层（核）通过并联的方式结合在一起

3 层普通的卷积层 + 3 组子网络（分别包含 2、5、2 个 Inception 模块）
+ 平均池化层和全连接层



Christian Szegedy, et al. "Going deeper with convolutions." Proceedings of the IEEE conference on computer vision and pattern recognition 2015

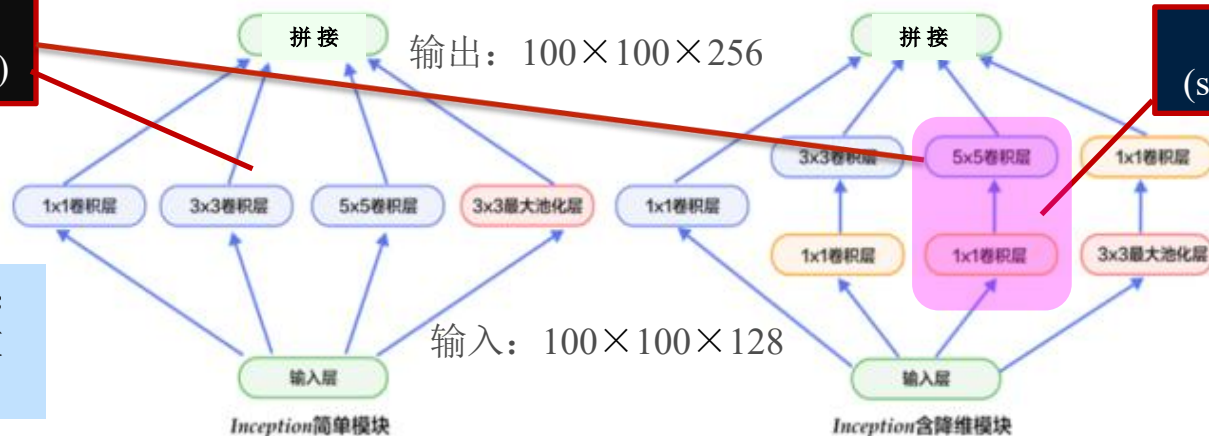
GoogleNet 的主要特点

- NIN 思想提高了特征表达能力：多尺度
- InceptionV1 模块，控制了计算量和参数量：网络的深度和宽度高效率地扩充
- 1×1 卷积：跨通道降维（即减少通道数）+ 用很小的计算量增加特征变换和非线性化
- top-5 错误率 6.67%，低于 AlexNet 的一半，Inception V1 有 22 层深，但其参数量（5M）仅为 AlexNet 参数量（60M）的 1/12

Inception 模块作用：
1) 多个尺寸上同时进行卷积再聚合
2) 使用 1×1 的卷积来进行升降维；

256个 5×5 卷积
(stride=1, pad=2)

简单模块(无偏置):
 5×5 卷积层的参数
= $256 \times 5 \times 5 \times 128$



32个 1×1 卷积
(stride=1, pad=0)

$100 \times 100 \times 32$

含降维模块: 1×1 卷积层和 5×5 卷积层的总参数量= $32 \times 1 \times 1 \times 128 + 256 \times 5 \times 5 \times 32$, 约减少4倍

type	patch size/ stride	output size	depth	#1×1	#3×3 reduce	#3×3	#5×5 reduce	#5×5	pool proj	params	ops
convolution	7×7/2	112×112×64	1							2.7K	34M
max pool	3×3/2	56×56×64	0								
convolution	3×3/1	56×56×192	2		64	192				112K	360M
max pool	3×3/2	28×28×192	0								
inception (3a)		28×28×256	2	64	96	128	16	32	32	159K	128M
inception (3b)		28×28×480	2	128	128	192	32	96	64	380K	304M
max pool	3×3/2	14×14×480	0								
inception (4a)		14×14×512	2	192	96	208	16	48	64	364K	73M
inception (4b)		14×14×512	2	160	112	224	24	64	64	437K	88M
inception (4c)		14×14×512	2	128	128	256	24	64	64	463K	100M
inception (4d)		14×14×528	2	112	144	288	32	64	64	580K	119M
inception (4e)		14×14×832	2	256	160	320	32	128	128	840K	170M
max pool	3×3/2	7×7×832	0								
inception (5a)		7×7×832	2	256	160	320	32	128	128	1072K	54M
inception (5b)		7×7×1024	2	384	192	384	48	128	128	1388K	71M
avg pool	7×7/1	1×1×1024	0								
dropout (40%)		1×1×1024	0								
linear		1×1×1000	1							1000K	1M
softmax		1×1×1000	0								

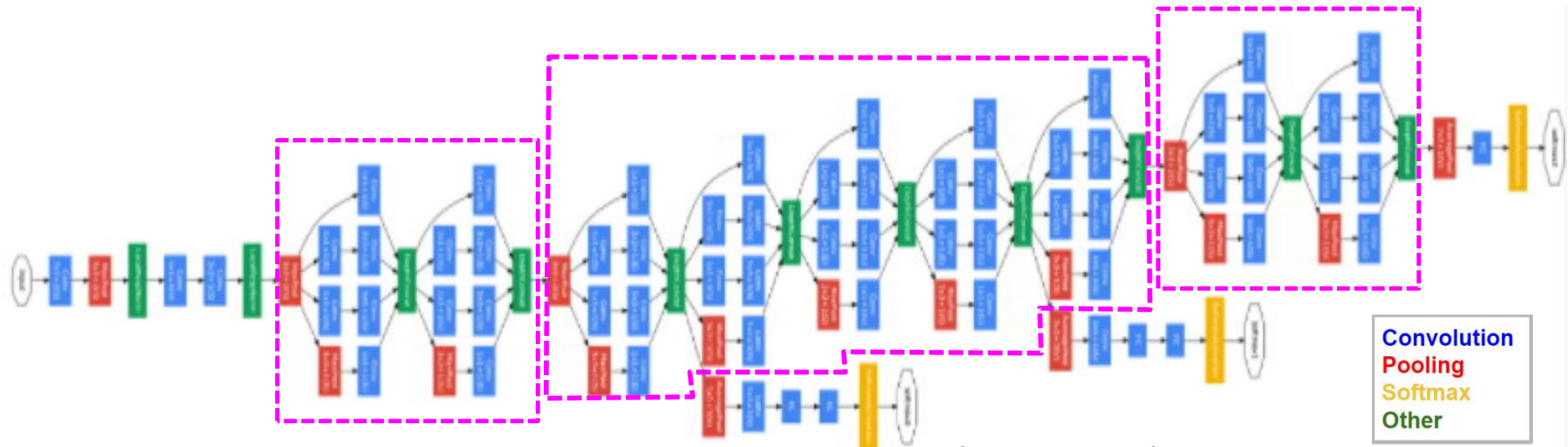
22层

Table 1: GoogLeNet incarnation of the Inception architecture

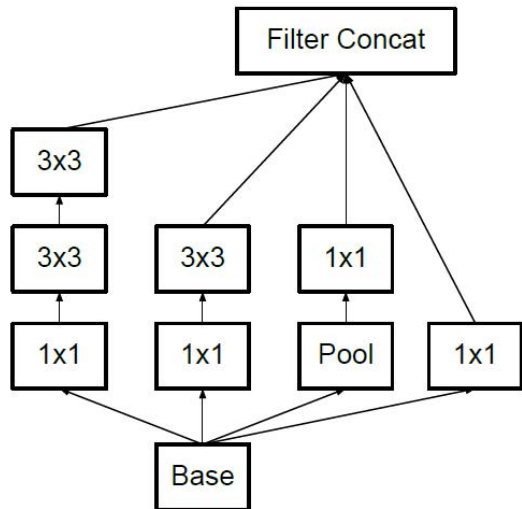
GoogLeNet 的主要特点

- 由多组 Inception 模块堆叠而成
- 在网络的最后采用了均值池化层来替代传统的多层全连接层
- 在池化层后接了一层全连接层以映射到类别数
- 考虑到网络中间层特征也很有判别性，在中间层添加了两个辅助分类器，用于在反向传播中增强梯度同时增强正则化，而整个网络的损失函数由这三个分类器的损失加权求和得到。

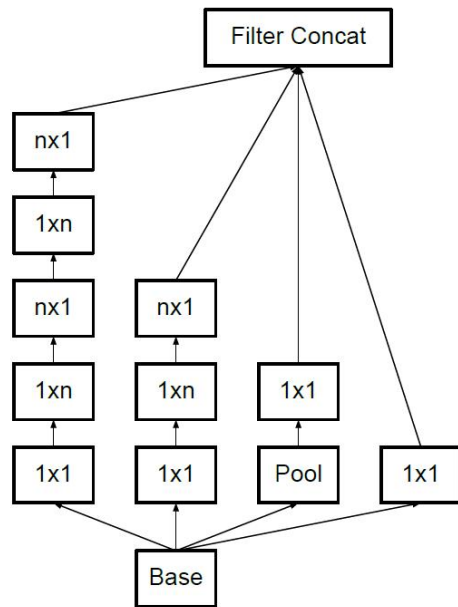
Google Inception: 包括inceptionV1（22层）、inceptionV2、inceptionV3（47层）、inceptionV4等



- ▶ 用多层小卷积核替换大卷积核，以减少计算量和参数量。
- ▶ 使用两层 3×3 的卷积来替换 v1 中的 5×5 的卷积
- ▶ 使用连续的 $n \times 1$ 和 $1 \times n$ 来替换 $n \times n$ 的卷积。



Inception modules where each 5×5 convolution is replaced by two 3×3 convolution, as suggested by principle [3] of Section [2].

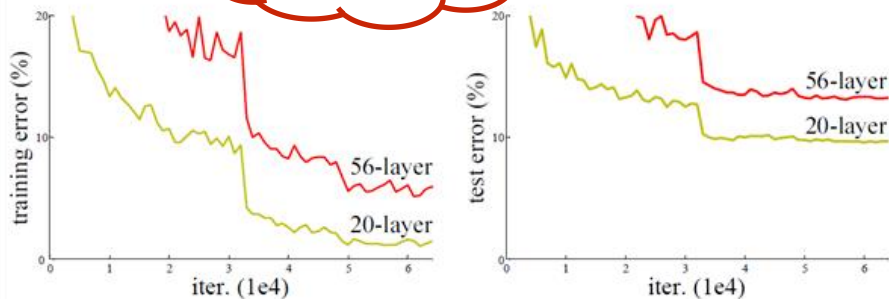


Inception modules after the factorization of the $n \times n$ convolutions. In our proposed architecture, we chose $n = 7$ for the 17×17 grid. (The filter sizes are picked using principle [3])

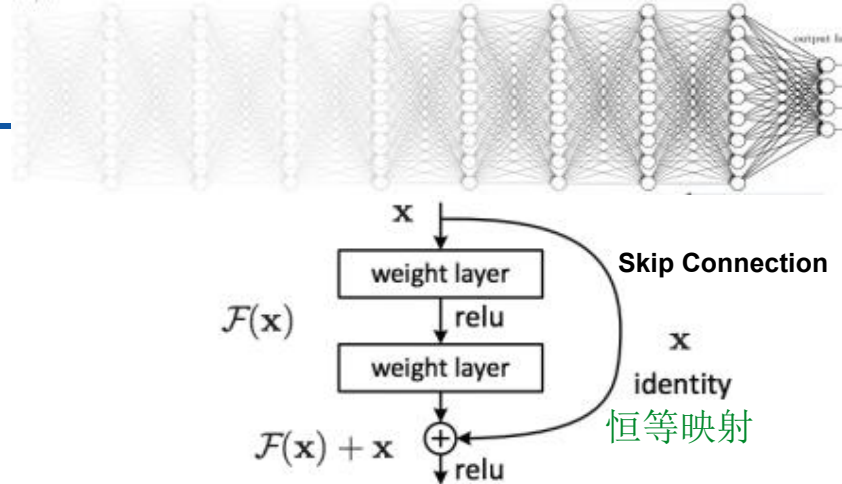
ResNet (Residual Network)

- 微软何凯明提出: CVPR2016 最佳论文奖
- 获得 2015 年 ILSVRC 挑战冠军图像分类、图像物体定位和图像物体检测比赛冠军: 6.7% - > 3.57%
- 更深: 152 层
- 针对网络加深导致准确度下降的问题, 提出**残差模块**: 与从头开始学习一个恒等变换相比, 学习如何使 $F(x) = 0$ 并使输出仍为 x 要更容易

深度网络的退化问题



20层与56层网络在CIFAR-10上的误差



每个**残差模块**包含两条路径,

1. 一条路径借鉴 Highway Network 思想, 在旁侧专门开辟一个通道使得输入 x 可以直达输出;
2. 一条路径对输入特征做 2 到 3 次卷积操作得到该特征对应的**残差** $F(x)$

最后再将两条路径上的输出相加, 优化的目标由原来的拟合输出 $H(x)$ 变成输出和输入的差 $F(x) = H(x) - x$

- ◆ 虽然学习出的结果是一样的, 但是学习难度是不同的.
- ◆ 恒等 **shortcut** 不增加额外参数也不增加计算复杂度.

$$y_l = h(x_l) + F(x_l, W_l)$$

$$x_{l+1} = f(y_l)$$

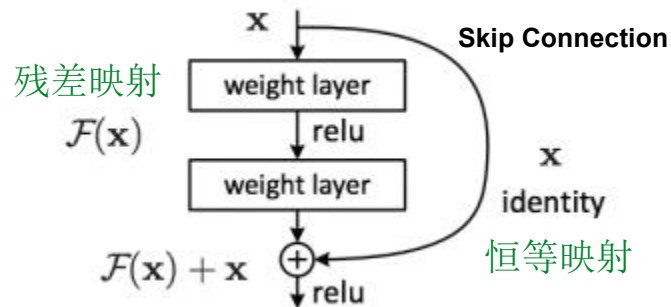
其中 x_l 和 x_{l+1} 分别表示的是第 l 个残差单元的输入和输出，注意每个残差单元一般包含多层结构。 F 是残差函数，表示学习到的残差，而 $h(x_l) = x_l$ 表示恒等映射， f 是ReLU激活函数。基于上式，我们求得从浅层 l 到深层 L 的学习特征为：

$$x_L = x_l + \sum_{i=l}^{L-1} F(x_i, W_i)$$

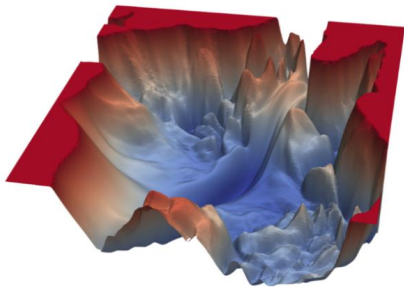
利用链式规则，可以求得反向过程的梯度：

$$\frac{\partial loss}{\partial x_l} = \frac{\partial loss}{\partial x_L} \cdot \frac{\partial x_L}{\partial x_l} = \frac{\partial loss}{\partial x_L} \cdot \left(1 + \frac{\partial}{\partial x_l} \sum_{i=l}^{L-1} F(x_i, W_i) \right)$$

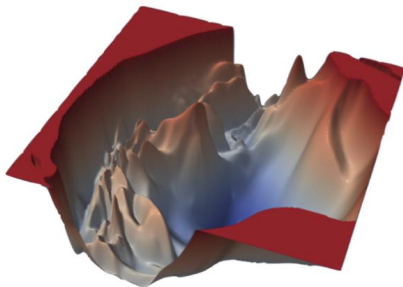
式子的第一个因子 $\frac{\partial loss}{\partial x_L}$ 表示的损失函数到达 L 的梯度，小括号中的1表明短路机制可以无损地传播梯度，而另外一项残差梯度则需要经过带有weights的层，梯度不是直接传递过来的。残差梯度不会那么巧全为-1，而且就算其比较小，有1的存在也不会导致梯度消失。所以残差学习会更容易。要注意上面的推导并不是严格的证明。



VGG-56



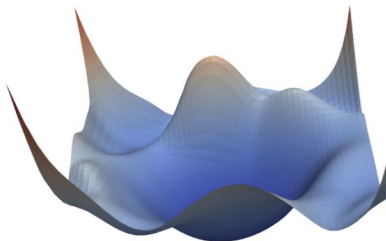
VGG-110



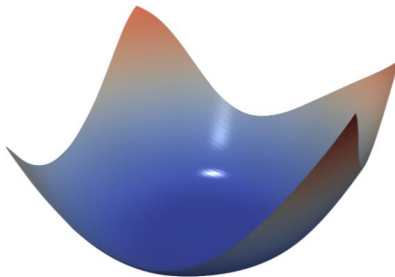
without skip connections

损失曲面的可视化方法：通过沿着两个随机方向对参数进行扰动，计算损失值，并绘制等高线图。

Resnet-56



Densenet-121



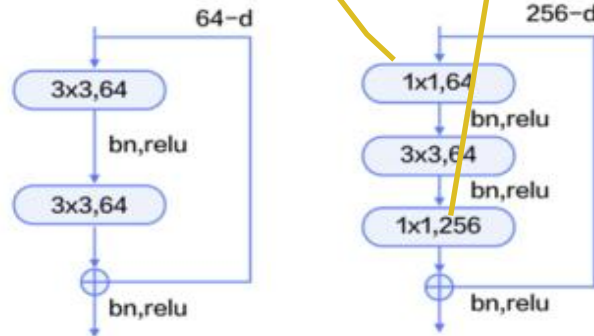
with skip connections

Li H, Xu Z, Taylor G, et al. Visualizing the loss landscape of neural nets[C]//Advances in Neural Information Processing Systems. 2018: 6389-6399.

ResNet 的主要特点

➤ 残差模块:

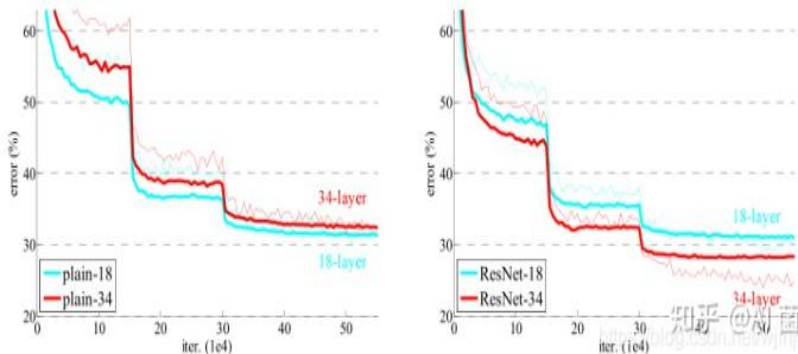
- 左侧是基本模块连接方式, 由两个输出通道数相同的 3×3 卷积层组成;
 - 右侧是瓶颈模块 (Bottleneck) 连接方式
- 随着网络层数不断加深, 训练误差会随着层数增大反而逐渐减小, 训练收敛速度较快



基本残差模块

瓶颈残差模块

• 采取三层块结构以减小计算量和参数量



在ImageNet上的训练结果。细线表示训练误差, 粗线表示验证误差。

左: 18和34层的普通网络。 右: 18和34层的ResNet。

普通的网络出现退化现象, 但ResNet很好地解决了退化问题

method	top-1 err.	top-5 err.
VGG [41] (ILSVRC'14)	-	8.43 [†]
GoogLeNet [44] (ILSVRC'14)	-	7.89
VGG [41] (v5)	24.4	7.1
PRReLU-net [13]	21.59	5.71
BN-inception [16]	21.99	5.81
ResNet-34 B	21.84	5.71
ResNet-34 C	21.53	5.60
ResNet-50	20.74	5.25
ResNet-101	19.87	4.60
ResNet-152	19.38	4.49

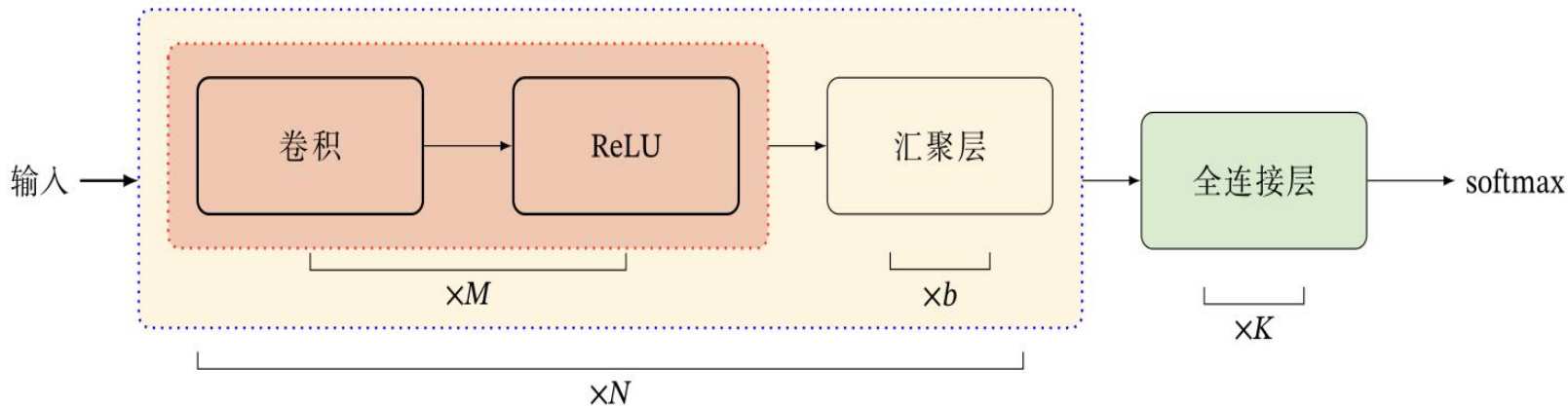
模型名	AlexNet	VGG	GoogLeNet	ResNet
初入江湖	2012	2014	2014	2015
层数	8	19	22	152
Top-5错误	16.4%	7.3%	6.7%	3.57%
Data Augmentation	+	+	+	+
Inception(NIN)	-	-	+	-
卷积层数	5	16	21	151
卷积核大小	11,5,3	3	7,1,3,5	7,1,3
全连接层数	3	3	1	1
全连接层大小	4096,4096,1000	4096,4096,1000	1000	1000
Dropout	+	+	+	+
Local Response Normalization	+	-	+	-
Batch Normalization	-	-	-	+

层数：卷积层和全连接层的个数，没有参数的池化层不包括在内

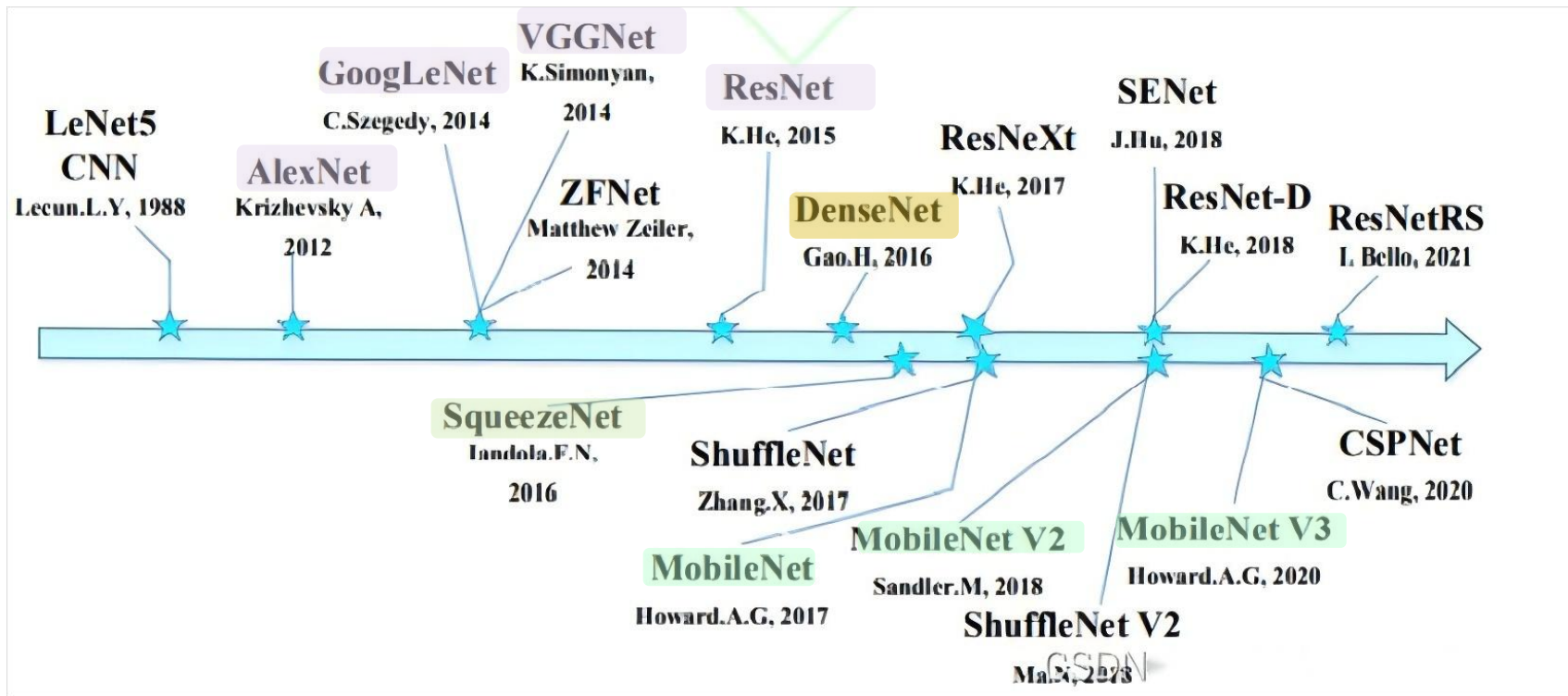
Model	Model Size(MB)	Million Mult-Adds	Million Parameters
AlexNet ^[1]	>200	720	60
VGG16 ^[2]	>500	15300	138
GoogleNet ^[3]	~50	1550	6.8
Inception-v3 ^[4]	90-100	5000	23.2

152层的残差网络
参数量：约60.2M
计算量：11.3
billion FLOPs，比
16层的VGG少

- 卷积神经网络是由卷积层、池化层（汇聚层）、全连接层堆叠而成。
 - 趋向于小卷积、大深度
 - 趋向于全卷积
- 典型结构



- 一个卷积块为连续 M 个卷积层和 b 个汇聚层（ M 通常设置为2 ~ 5， b 为0或1）。
- 一个卷积神经网络中可以堆叠 N 个连续的卷积块，然后在接着 K 个全连接层（ N 的取值区间比较大，比如1 ~ 100或者更大； K 一般为0 ~ 2）。



- **DenseNet**（CVPR 2017最佳论文，ResNet的扩展，包含1000层的架构）：进一步提高ImageNet图像分类的准确度。
- **SqueezeNet（2016）、MobileNet（2017）**等轻量级架构：能提供合理的准确度，相比于VGG、ResNet和Inception等架构，这些架构规模很小。
- **NASNet（2017）**：Google的AutoML系统生成的架构，在ImageNet的top-5错误率为3.8%。
- **PNAS（2017）**：Google的另一个自动生成架构——渐进式神经架构搜索，没有使用强化学习（RL）或遗传算法（GA），而是使用了基于序列模型的优化（SMBO）策略，在其中按复杂度逐渐增大的顺序搜索架构，同时学习一个用于引导该搜索的代理函数（surrogate function），类似于A*搜索。

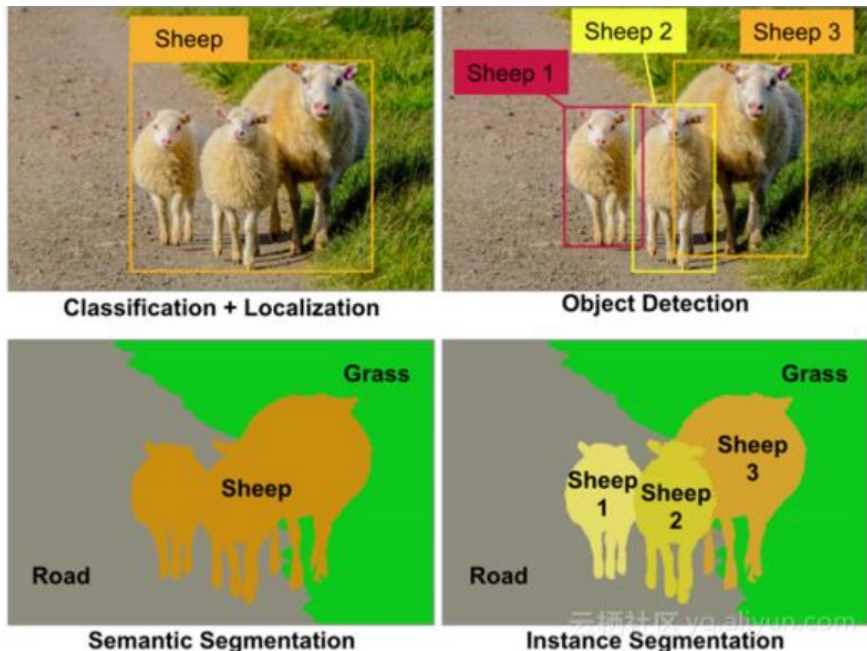
什么是胶囊网络？<https://www.bilibili.com/video/BV1ty4y1r7M8>



```
from torchvision import models
import torch
print(dir(models))
```

```
['AlexNet', 'DenseNet', 'GoogLeNet', 'GoogLeNetOutputs', 'Inception3', 'InceptionOutputs',
'MNASNet', 'MobileNetV2', 'ResNet', 'ShuffleNetV2', 'SqueezeNet', 'VGG', ..., 'alexnet', 'densenet',
'densenet121', 'densenet161', 'densenet169', 'densenet201', 'detection', 'googlenet', 'inception',
'inception_v3', 'mnasnet', 'mnasnet0_5', 'mnasnet0_75', 'mnasnet1_0', 'mnasnet1_3', 'mobilenet',
'mobilenet_v2', 'quantization', 'resnet', 'resnet101', 'resnet152', 'resnet18', 'resnet34', 'resnet50',
'resnext101_32x8d', 'resnext50_32x4d', 'segmentation', 'shufflenet_v2_x0_5', 'shufflenet_v2_x1_0',
'shufflenet_v2_x1_5', 'shufflenet_v2_x2_0', 'shufflenetv2', 'squeezenet', 'squeezenet1_0',
'squeezenet1_1', 'utils', 'vgg', 'vgg11', 'vgg11_bn', 'vgg13', 'vgg13_bn', 'vgg16', 'vgg16_bn', 'vgg19',
'vgg19_bn', 'video', 'wide_resnet101_2', 'wide_resnet50_2']
```

案例参考https://gitee.com/mindspore/models/blob/master/README_CN.md



典型的卷积神经网络(CNN)架构，例如用于分类的**ResNet**、**Vision Transformer (ViT)**，用于目标检测的**Faster R-CNN**、**YOLO v9**，用于语义分割的**FCN**、**Unet**、**SegNet**、**Deeplabv3**，用于实例分割的**Mask R-CNN**，交互式图像分割**SAM**等

图像处理、计算机视觉领域...



垃圾分类

通过图像分类技术
判断垃圾所属类别。



车牌识别

通过图像检测技术
识别车辆车牌信息。



图像搜索

通过图像搜索技术
给用户提供更多检索信息。

二维卷积用于图像处理、计算机视觉



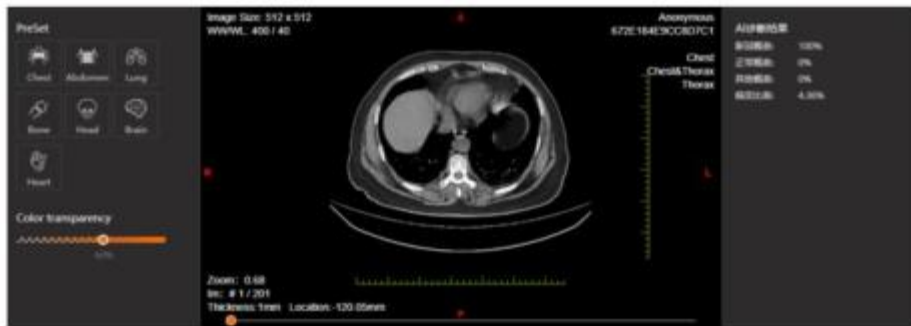
人脸检测与人体检测



商品推荐

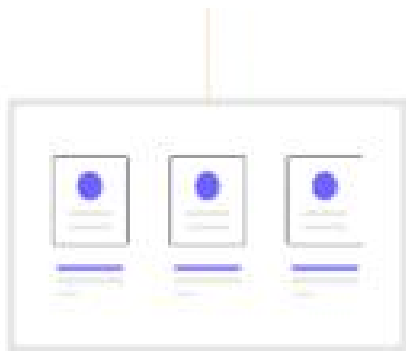


图像色彩增强



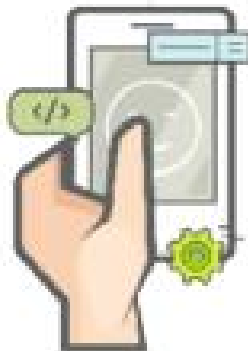
医疗应用

序列模型、自然语处理模型...



文本分类

将不同领域的文本
内容进行归类存放。



语音识别

通过输入语音进行
识别并转为文本内容。



机器翻译

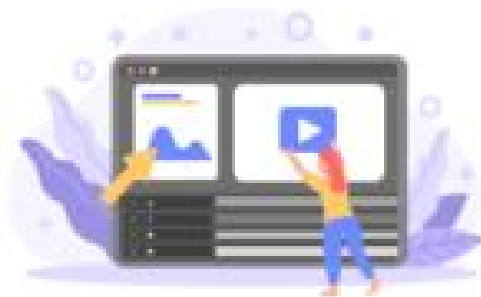
通过计算机将一种语言
转为另外一种语言。

一维卷积用于序列模型和自然语言处理模型



医学领域

CT影像的检测，协助医生快速准确的对患者病患进行初步判断。



视频处理领域

识别出监控视频中出现的**人物**，以及在家庭监控视频中自动识别是否出现**婴儿或宠物**，可用于安防领域，守护社会稳定及家庭安全。



农业病虫害识别

科研工作者已经将其广泛运用于病虫害识别检测、植物和农作物的识别、杂草的检测与分类等多项研究。

三维卷积的应用



1. 在卷积神经网络中，感受野的含义是什么？卷积神经网络的关键技术有哪些？卷积层和池化层的作用分别是什么？卷积层的参数量、连接数怎么确定？
2. 在卷积操作中，理解单通道卷积和多通道卷积的区别。输出特征图的通道数、大小怎么确定？填充的作用是什么？卷积操作的参数量、计算量如何确定？
3. VGG中，使用3个 3×3 卷积核代替 7×7 卷积核，使用2个 3×3 卷积核代替 5×5 卷积核的好处有哪些？
4. GoogLeNet中，Inception 模块作用是什么？理解含降维的Inception模块和简单的Inception模块的参数量分析
5. ResNet中，skip connections（跳接）的作用是什么？理解基本残差模块和瓶颈残差模块的参数量分析，以及 1×1 卷积的作用。
6. 在实际应用中，如何根据视觉任务，选择合适的卷积神经网络？